Zona Python Bloque 01

Temario aula

Table of contents

1. Zona Python 3
   1.1 Referencias 3
2. Bl. 01 4
   2.1 Introducción a Python 4
   2.2 Estructuras control 13
   2.3 Estructuras Datos 28
   2.4 Ejercicios I 46
   2.5 Módulos/funciones 54
   2.6 Introducción a Python - Excepciones 60
   2.7 Ejercicios II 65
   2.8 Prácticas 69
3. Tools 74
   3.1 Introducción a Python - Linters 74
   3.2 Introducción a Python - Introducción al testing 80
   3.3 Python: Calidad y documentación del código 87
   3.4 Depuración de scripts Python en Linux con VS Code 95
4. Utiles 103
   4.1 Python y venv 103
   4.2 Introducción a `logging` 109
   4.3 Introducción a Python: argumentos en línea de comandos 111
   4.4 Pyinstaller 115

1. Zona Python

Apuntes y complementos Módulo sobre Python

1.1 Referencias

- Tutorial Python de W3Schools
- Curso online en Netacad: Fundamentos de Python 1

2. Bl. 01

2.1 Introducción a Python

En estos ejemplos se presentan algunos de los elementos básicos que utilizaremos continuamente al programar en Python: **variables**, **tipos de datos**, **cadenas de texto**, **valores booleanos** y **conversión de tipos**.

2.1.1 1. Entorno de trabajo

- Sistema operativo Linux con Python ya instalado
- Editor de texto Visual Studio Code
- Trabajo en terminal

2.1.2 2. Variables y tipos de datos

Una **variable** permite guardar un valor para utilizarlo posteriormente. En Python no es necesario indicar de antemano el tipo de una variable: este se determina a partir del valor que le asignamos.

2.1. Código completo

```
# Variables, asignación (=) y tipos (int, float, str...)

# Asignación
saldo_total = 254.55  # float
email = 'ana@local.com'  # str

print("saldo_total") # imprime cadena
print(saldo_total)   # imprime valor de la variable
  print(email)   # error indentación: el programm se detiene

# Imprime tipo de las variables usando type(variable)
print("tipo de email: ", type(email)  )
print("tipo de saldo_total: ", type(saldo_total)  )
# print puede recibir varias expresiones separadas por comas

# Variables pueden cambiar de tipo
#  NO es algo recomdable
email = 4  # pasa de string a integer
print( "Tipo email: ", type(email)  )
print("El email es ", email, " y el saldo es ", saldo_total, "\n")
```

2.2. Asignar valores a variables

Las primeras instrucciones crean dos variables:

```
saldo_total = 254.55
email = 'ana@local.com'
```

`saldo_total` contiene un número decimal, por lo que su tipo es `float`. `email` contiene texto, por lo que su tipo es `str`.

El símbolo `=` es el **operador de asignación**: guarda el valor situado a la derecha en la variable situada a la izquierda.

> **Asignación y comparación no son lo mismo**
>
> En Python, `=` se utiliza para **asignar** un valor. Más adelante veremos que `==` se utiliza para **comparar** dos valores.

2.3. Imprimir texto o imprimir una variable

Estas dos instrucciones parecen muy similares, pero no hacen lo mismo:

```
print("saldo_total")
print(saldo_total)
```

La primera imprime literalmente el texto:

```
saldo_total
```

La segunda consulta el contenido de la variable e imprime:

```
254.55
```

Las comillas son, por tanto, importantes: `"saldo_total"` es una cadena de texto, mientras que `saldo_total` es el nombre de una variable.

2.4. Un error de indentación

El ejemplo contiene intencionadamente esta línea:

```
  print(email)
```

Hay espacios al comienzo de la línea sin que exista ningún bloque que justifique esa indentación. Python considera la indentación parte de la sintaxis y detendrá el programa con un `IndentationError`.

Mientras ese error exista, las instrucciones posteriores **no llegarán a ejecutarse**.

La línea correcta sería:

```
print(email)
```

> **La indentación importa**
>
> En Python no utilizamos la indentación solo para que el programa quede más bonito. Sirve para indicar qué instrucciones pertenecen a determinados bloques de código.

2.5. Consultar el tipo de un dato

La función `type()` permite saber de qué tipo es un valor o una variable:

```
print("tipo de email: ", type(email))
print("tipo de saldo_total: ", type(saldo_total))
```

Una vez corregido el error anterior, veremos que inicialmente:

- `email` es de tipo `str`.
- `saldo_total` es de tipo `float`.

Además, `print()` puede recibir varios valores separados por comas:

```
print("El email es ", email, " y el saldo es ", saldo_total)
```

2.6. Una variable puede cambiar de tipo

Python permite hacer esto:

```
email = 4
```

Aunque antes `email` contenía una cadena, ahora contiene un entero (`int`).

Es posible hacerlo, pero normalmente **no es una buena práctica** reutilizar una variable para almacenar información de naturaleza completamente diferente. El nombre `email`, además, deja de describir correctamente lo que contiene.

2.7. Posibles mejoras

- Corregir la indentación de `print(email)`.
- Evitar que una variable llamada `email` termine almacenando el número `4`.
- Utilizar nombres de variables que describan claramente su contenido.
- Mantener un estilo uniforme en los espacios y llamadas a `print()`.

2.1.3 3. Trabajo con cadenas de texto

Las cadenas (`str`) representan texto. Python proporciona muchas operaciones para mostrarlas, transformarlas, buscar contenido en ellas y recorrer sus caracteres.

3.1. Código completo

```
# Ejemplo uso format en print
nombre = "Alfredo Zumberg"
edad = 23
altura = 1.915
print(nombre, "su edad es", edad, "y altura", altura)
# Uso de format con cadenas
print(f"{nombre} su edad es {edad} y altura {altura:.1f}")


# Algunos métodos de string

saludo = "Hola Mundo"
print("Valor de saludo:", saludo)

print("Resultado de saludo.lower():", saludo.lower())
print("Valor de saludo:", saludo)


# No modifica cadena original.
# Si queremos modificar hace falta asignación
saludo = saludo.lower()
print("Valor de saludo:", saludo)

# Replace, lo mismo..
print('Resultado de saludo.replace("mundo", "universo"):' , saludo.replace("mundo", "universo"))
print("Valor de saludo:", saludo)

# el tipo str se puede tratar como array o lista:
saludo ="¡Hola mundo!"
print("Primer carácter:", saludo[0])
print("Segundo carácter:",saludo[1])
print("Longitud saludo:", len(saludo))

# Buscar en un str: in y not in
texto = "The best thiNgs  life are free!"
busca = "FREE"

if busca.lower() in texto.lower():
    print(f"{busca} sí está en {texto}")
else:
    print(f"{busca} NO está en {texto}")

# o en negativo y sin ser case sensitve
if busca not in texto:
    print(f"{busca} NO está en {texto}")
else:
    print(f"{busca} sí está en {texto}")


# Concatenar str
nombre = "Raquel"
apellidos = "García Ross"
print(nombre + " " + apellidos)  # el operador + con cadenas concatena

nombreCompleto = apellidos + ", " + nombre
print(nombreCompleto)

# Extraer parte del string: slice

# La sintaxis del slicing en Python es string[inicio:fin:paso]:
#   - inicio: Por defecto empieza desde el extremo del string.
#   - fin: Por defecto recorre hasta el otro extremo.
#   - paso: por defecto 1.

saludo = "Hola Mundo"
print(saludo[:4])  # "Hola"
print(saludo[5:])  # "Mundo"
print(saludo[::2]) # de dos en dos => "Hl ud"
```

```
# Para invertir una cadena se usa salto negativo
print(saludo[::-1]) # en order inverso => "odnuM aloH"



# Se muestra ahora el uso de cadenas con bucle for

nombre = "www.aulas.local"
# a) Recorrer cada caracter de un str
for x in nombre:
    print(x)

# b) Recorrer cada caracter de un str
# usando el índice o posición
# se introduce el uso de range()
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")


# Ejemplo de ejecución de range
# >>> for i in range(5):
# ...     print(i)
# ...
# 0
# 1
# 2
# 3
# 4
# >>>
```

3.2. Mostrar valores con `print()` y f-strings

Podemos imprimir varios valores separándolos mediante comas:

```
print(nombre, "su edad es", edad, "y altura", altura)
```

Otra posibilidad muy cómoda son las **f-strings**:

```
print(f"{nombre} su edad es {edad} y altura {altura:.1f}")
```

Todo lo que aparece entre `{}` se sustituye por su valor.

En el caso de:

```
{altura:.1f}
```

se solicita mostrar el número decimal con **una cifra después del punto decimal**.

3.3. Métodos de las cadenas

Una cadena dispone de métodos que permiten obtener nuevas cadenas transformadas.

Por ejemplo:

```
saludo.lower()
```

devuelve una versión en minúsculas.

Es importante observar que:

```
print(saludo.lower())
print(saludo)
```

no modifica el contenido original de `saludo`.

Para conservar el nuevo valor debemos asignarlo:

```
saludo = saludo.lower()
```

Algo parecido sucede con `replace()`:

```
saludo.replace("mundo", "universo")
```

devuelve una cadena nueva en la que se ha realizado la sustitución, pero no modifica automáticamente `saludo`.

> **Las cadenas son inmutables**
>
> Una cadena no se modifica internamente. Operaciones como `lower()` o `replace()` producen una **nueva cadena**. Si queremos conservarla, tendremos que asignarla a una variable.

3.4. Acceder a caracteres y obtener la longitud

Podemos acceder a los caracteres de una cadena mediante su posición:

```
saludo = "¡Hola mundo!"
print(saludo[0])
print(saludo[1])
```

Las posiciones comienzan en **0**:

- `saludo[0]` es `¡`.
- `saludo[1]` es `H`.

La función `len()` devuelve el número de caracteres:

```
len(saludo)
```

Aunque en el comentario original se compara un `str` con un array o una lista, es más preciso decir que una cadena es una **secuencia de caracteres**. Podemos acceder a sus posiciones de forma parecida a una lista, pero una cadena no se puede modificar carácter a carácter porque es inmutable.

3.5. Buscar texto con `in` y `not in`

El operador `in` permite comprobar si un texto aparece dentro de otro:

```
if busca.lower() in texto.lower():
```

En el ejemplo convertimos ambas cadenas a minúsculas antes de compararlas. De esta forma no importa que una aparezca como `"FREE"` y otra como `"free"`.

`not in` comprueba lo contrario:

```
if busca not in texto:
```

> **Atención al segundo ejemplo**
>
> El comentario indica que esta segunda búsqueda tampoco distingue mayúsculas y minúsculas, pero el código **sí las distingue**, porque compara directamente `busca` con `texto`.
>
> Si quisiéramos una búsqueda negativa sin distinguir mayúsculas y minúsculas, podríamos escribir:
>
> ```
> if busca.lower() not in texto.lower():
>     print(f"{busca} NO está en {texto}")
> ```

3.6. Concatenar cadenas

El operador `+` une cadenas:

```
nombre = "Raquel"
apellidos = "García Ross"

print(nombre + " " + apellidos)
```

También podemos construir y guardar una nueva cadena:

```
nombreCompleto = apellidos + ", " + nombre
```

El resultado sería:

```
García Ross, Raquel
```

3.7. Extraer partes de una cadena: slicing

Python permite obtener fragmentos mediante la sintaxis:

```
cadena[inicio:fin:paso]
```

Por ejemplo:

```
saludo = "Hola Mundo"

print(saludo[:4])
print(saludo[5:])
print(saludo[::2])
```

- `saludo[:4]` toma desde el principio hasta la posición 4, sin incluirla.
- `saludo[5:]` toma desde la posición 5 hasta el final.
- `saludo[::2]` recorre la cadena de dos en dos.

También podemos utilizar un paso negativo:

```
saludo[::-1]
```

Esto recorre la cadena en sentido contrario y permite invertirla.

3.8. Recorrer una cadena con `for`

Una cadena puede recorrerse carácter a carácter:

```
for x in nombre:
    print(x)
```

En cada vuelta del bucle, `x` contiene uno de los caracteres.

También podemos recorrer las posiciones:

```
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")
```

Aquí se combinan tres elementos:

- `len(nombre)` obtiene la longitud.
- `range(...)` genera las posiciones desde `0`.
- `nombre[i]` obtiene el carácter situado en cada posición.

Por ejemplo:

```
range(5)
```

genera los valores:

```
0
1
2
3
4
```

El `5` no se incluye.

3.9. Posibles mejoras

- Utilizar nombres en estilo `snake_case`, por ejemplo `nombre_completo` en lugar de `nombreCompleto`.
- Corregir el comentario de la segunda búsqueda con `not in`, ya que el código mostrado sí distingue mayúsculas y minúsculas.
- Hablar de las cadenas como **secuencias de caracteres** en vez de considerarlas arrays o listas.
- Cuando solo queremos recorrer los caracteres, `for caracter in cadena` es normalmente más sencillo que usar posiciones.

2.1.4 4. Variables lógicas o booleanas

El tipo `bool` representa valores lógicos. Solo existen dos:

```
True
False
```

4.1. Código completo

```
# lógica o booleana
autorizado = False    # el otro valor posible True
print( type(autorizado)  )


# ejemplo de uso de vaiable lógica
if autorizado==True:  # autorizado==True <=> autorizado
    print("Puede pasar")
    print("resto del bloque positivo")
else:
    print("No puede pasar")
    print("resto del bloque negativo")
```

4.2. El tipo `bool`

La variable:

```
autorizado = False
```

contiene un valor booleano.

Podemos comprobar su tipo:

```
print(type(autorizado))
```

4.3. Utilizar un booleano en una condición

El ejemplo utiliza:

```
if autorizado == True:
```

Si la condición es verdadera, se ejecutan las instrucciones indentadas debajo del `if`. En caso contrario, se ejecuta el bloque asociado a `else`.

Como `autorizado` vale `False`, se ejecutará:

```
print("No puede pasar")
print("resto del bloque negativo")
```

La indentación indica qué instrucciones forman parte de cada bloque.

4.4. Una forma más sencilla

No es necesario comparar explícitamente un booleano con `True`.

En lugar de:

```
if autorizado == True:
```

podemos escribir:

```
if autorizado:
```

Ambas formas expresan la misma condición, pero la segunda es más habitual y clara en Python.

4.5. Posibles mejoras

El ejemplo podría escribirse de forma más sencilla así:

```
autorizado = False

if autorizado:
    print("Puede pasar")
    print("Resto del bloque positivo")
else:
    print("No puede pasar")
    print("Resto del bloque negativo")
```

2.1.5 5. Conversión de tipos

Los datos introducidos mediante `input()` se obtienen como cadenas de texto. Si queremos realizar operaciones numéricas con ellos, tendremos que convertirlos.

5.1. Código completo

```
# conversión de tipos

# Ejemplo sin cast que no "funciona"
saldo = input("saldo: ")
incremento = input("incremento: ")

total = saldo + incremento
print("Valor de total sin cast: ", total)
print(type(saldo))
print(type(incremento))
"""
 El operador + en cadenas concatena
"""

# Se intenta ahora haciendo cast de las cadenas
saldo = float(saldo)
incremento = float(incremento)
total = saldo + incremento
print("Valor de total: ", total)

"""
 => Convertir a números las lecturas antes de operar con ellas
"""
```

5.2. `input()` devuelve una cadena

Estas instrucciones leen datos escritos por el usuario:

```
saldo = input("saldo: ")
incremento = input("incremento: ")
```

Aunque escribamos números, el resultado de `input()` es de tipo `str`.

Por ejemplo, si escribimos:

```
saldo: 100
incremento: 25
```

internamente tendremos dos cadenas equivalentes a:

```
saldo = "100"
incremento = "25"
```

5.3. ¿Qué sucede al utilizar `+` ?

La operación:

```
total = saldo + incremento
```

no suma matemáticamente los valores. Como ambos son cadenas, `+` las **concatena**.

Con los valores anteriores obtendríamos:

```
10025
```

en vez de:

```
125
```

Las llamadas a `type()` permiten comprobar el motivo:

```
print(type(saldo))
print(type(incremento))
```

5.4. Convertir a `float`

Podemos convertir ambas cadenas a números decimales:

```
saldo = float(saldo)
incremento = float(incremento)
```

A partir de ese momento:

```
total = saldo + incremento
```

sí realiza una suma numérica.

Este proceso de convertir un valor de un tipo a otro suele denominarse **conversión de tipos** o, habitualmente, *casting*.

Una forma habitual

Si sabemos desde el principio que necesitamos un número, también podemos convertir directamente el resultado de `input()` :

```
saldo = float(input("Saldo: "))
incremento = float(input("Incremento: "))
```

5.5. ¿Y si el usuario no introduce un número?

La conversión:

```
float(saldo)
```

solo funciona si el contenido puede interpretarse como un número.

Por ejemplo, si el usuario introduce:

```
hola
```

Python producirá un `ValueError` .

Más adelante podremos controlar este tipo de situaciones validando los datos introducidos o tratando las excepciones.

2.2 Estructuras control

2.2.1 Introducción a Python

Las **estructuras selectivas** permiten que un programa tome decisiones y ejecute unas instrucciones u otras dependiendo de si se cumple una condición.

En Python utilizamos principalmente:

- `if`
- `if ... else`
- `if ... elif ... else`
- `match ... case`

1. Código completo

```python
## Selectiva simple (y  probar indentación)

edad = int(input("Edad: "))
print( type(edad) )

if  edad >= 18:  # selectiva simple
    print("Adulto")
    print("puede jugar") ## Error indentación
print("¡Hola Clase!")  # siempre,fuera if




# Ejemplo selectiva if-else y condición lógica con and

saldo = int(input("saldo: "))
edad = int(input("edad: "))

if (edad >= 18 and saldo > 0) or (saldo > 100000):
    print("Puede jugar")
else:
    print("no puede jugar")
print("Fin selectiva")  # siempre se muestra,fuera selectiva


# Ejemplo selectivas anidadas
a = int(input("valor de a:"))
b = 200

if b > a:
    print(f"{b} es mayor que {a}")
else:
    if b < a:
        print(f"{a} es mayor que {b}")
    else:
        print(f"{a} es igual a {b}")

# lo mismo con elif (else if), más legible/compacto
if b > a:
    print(f"{b} es mayor que {a}")
elif b < a:
    print(f"{a} es mayor que {b}")
else:
    print(f"{a} es igual a {b}")


"""
Selector multiple
 Cuando se quieren por ejemplo evaluar las opciones de un menú
 evita concatenar demasiados elif que hacen menos legible el código,
"""

opcion = int(input("Seleccione una opción: "))

match opcion:
    case 0:
        print("Ha seleccionado opción 0")
        print("procedemos opción 0")
    case 1:
        print("Ha seleccionado opción 1")
        print("procedemos opción 1")

    # otros case...

    case _:   #este es el caso por defecto
        print("Opción por defecto")
        print("No ha indicado ninguna opción contemplada")
```

2. Selectiva simple: `if`

Comenzamos leyendo la edad del usuario:

```python
edad = int(input("Edad: "))
print(type(edad))
```

Como `input()` devuelve una cadena, utilizamos `int()` para convertir el dato introducido a un número entero.

Después aparece la estructura:

```python
if edad >= 18:
    print("Adulto")
    print("puede jugar")
```

La expresión:

```
edad >= 18
```

es una **condición**. Su resultado solo puede ser `True` o `False`.

Si la edad es mayor o igual que 18, se ejecutan las instrucciones indentadas debajo del `if`.

Si no se cumple la condición, Python las salta y continúa ejecutando el programa.

2.1. LA IMPORTANCIA DE LA INDENTACIÓN

Observa:

```
if edad >= 18:
    print("Adulto")
    print("puede jugar")

print("¡Hola Clase!")
```

Las dos primeras llamadas a `print()` pertenecen al `if` porque están indentadas.

En cambio:

```
print("¡Hola Clase!")
```

está fuera del bloque, por lo que se ejecutará **siempre**, independientemente de la edad introducida.

> **La indentación delimita los bloques**
>
> En Python, las instrucciones que pertenecen a un `if`, `else`, bucle u otra estructura deben aparecer indentadas.

> **Sobre el comentario del ejemplo**
>
> En el script aparece el comentario `## Error indentación` junto a `print("puede jugar")`, pero esa línea tiene la misma indentación que la instrucción anterior y, tal como está escrita, pertenece correctamente al bloque `if`.
>
> Un error se produciría, por ejemplo, si una de las instrucciones tuviese una indentación incoherente.

3. `if-else`: elegir entre dos alternativas

Cuando queremos ejecutar una acción si se cumple una condición y otra diferente si no se cumple, utilizamos `if ... else`.

```
saldo = int(input("saldo: "))
edad = int(input("edad: "))

if (edad >= 18 and saldo > 0) or (saldo > 100000):
    print("Puede jugar")
else:
    print("no puede jugar")

print("Fin selectiva")
```

La estructura general es:

```
if condicion:
    # instrucciones si la condición es verdadera
else:
    # instrucciones si la condición es falsa
```

Solo se ejecutará uno de los dos bloques.

Después de ambos aparece:

```
print("Fin selectiva")
```

Como no está indentado dentro del `if` ni del `else`, se ejecuta siempre.

4. Condiciones compuestas con `and` y `or`

La condición anterior es más compleja:

```
(edad >= 18 and saldo > 0) or (saldo > 100000)
```

Aquí aparecen dos operadores lógicos.

4.1. OPERADOR `and`

Con `and`, las dos condiciones deben ser verdaderas:

```
edad >= 18 and saldo > 0
```

Se cumple solamente cuando:

- la persona tiene al menos 18 años;
- **y además** su saldo es mayor que 0.

4.2. OPERADOR `or`

Con `or`, basta con que se cumpla una de las condiciones.

La condición completa puede entenderse así:

```
(edad mínima Y saldo positivo)
O
(saldo mayor que 100000)
```

Por tanto, el programa permitirá continuar si se cumple cualquiera de esos dos grupos de condiciones.

> **Usar paréntesis ayuda a leer**
>
> Aunque en algunos casos Python podría interpretar correctamente la expresión sin paréntesis, utilizarlos ayuda a ver claramente qué condiciones se están agrupando.

5. Selectivas anidadas

Una estructura selectiva puede contener otra estructura selectiva.

El ejemplo compara dos números:

```
a = int(input("valor de a:"))
b = 200

if b > a:
    print(f"{b} es mayor que {a}")
else:
    if b < a:
        print(f"{a} es mayor que {b}")
    else:
        print(f"{a} es igual a {b}")
```

Primero se comprueba:

```
b > a
```

Si se cumple, ya sabemos que `b` es mayor.

Si no se cumple, entramos en el `else` y hacemos otra comprobación:

```
b < a
```

Si tampoco se cumple esta segunda condición, solo queda una posibilidad: ambos números son iguales.

Este tipo de construcción se denomina **selectiva anidada** porque hay un `if` dentro de otro bloque.

6. Simplificar con `elif`

Python dispone de `elif`, abreviatura de *else if*.

El ejemplo anterior puede escribirse así:

```
if b > a:
    print(f"{b} es mayor que {a}")
elif b < a:
    print(f"{a} es mayor que {b}")
else:
    print(f"{a} es igual a {b}")
```

Esta versión expresa exactamente las mismas posibilidades:

1. `b` es mayor que `a`.
2. `b` es menor que `a`.
3. Ambos son iguales.

Normalmente resulta más clara y evita añadir niveles innecesarios de indentación.

Python comprueba las condiciones de arriba hacia abajo y ejecuta el bloque correspondiente a la **primera que sea verdadera**.

Si ninguna se cumple, ejecuta el `else`.

7. Selector múltiple con `match`

Cuando queremos comparar un valor con varias opciones concretas podemos utilizar `match`.

```
opcion = int(input("Seleccione una opción: "))

match opcion:
    case 0:
        print("Ha seleccionado opción 0")
        print("procedemos opción 1")

    case 1:
        print("Ha seleccionado opción 1")
        print("procedemos opción 2")

    case _:
        print("Opción por defecto")
        print("No ha indicado ninguna opción contemplada")
```

Python toma el valor almacenado en `opcion` y busca un `case` que coincida.

Si el usuario introduce `0`, se ejecuta `case 0`. Si introduce `1`, se ejecuta `case 1`.

Para cualquier otro valor se ejecuta:

```
case _:
```

El guion bajo `_` representa aquí el **caso por defecto**.

7.1. `match` FRENTE A MUCHOS `elif`

Un menú también podría escribirse utilizando varios `elif`:

```
if opcion == 0:
    ...
elif opcion == 1:
    ...
elif opcion == 2:
    ...
else:
    ...
```

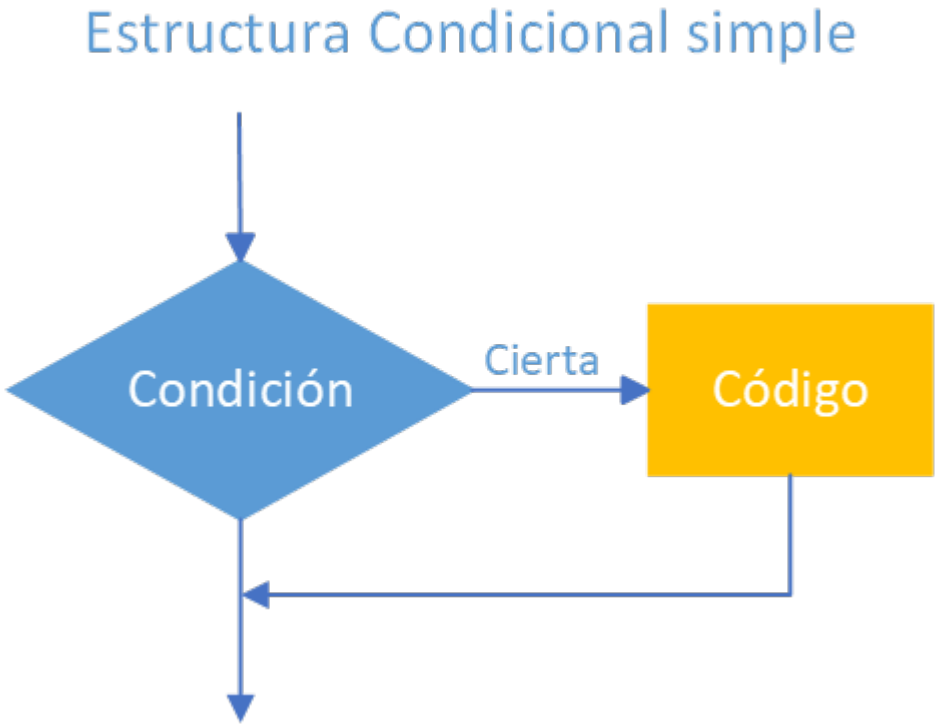
Cuando tenemos muchas opciones concretas, `match` puede resultar más legible.

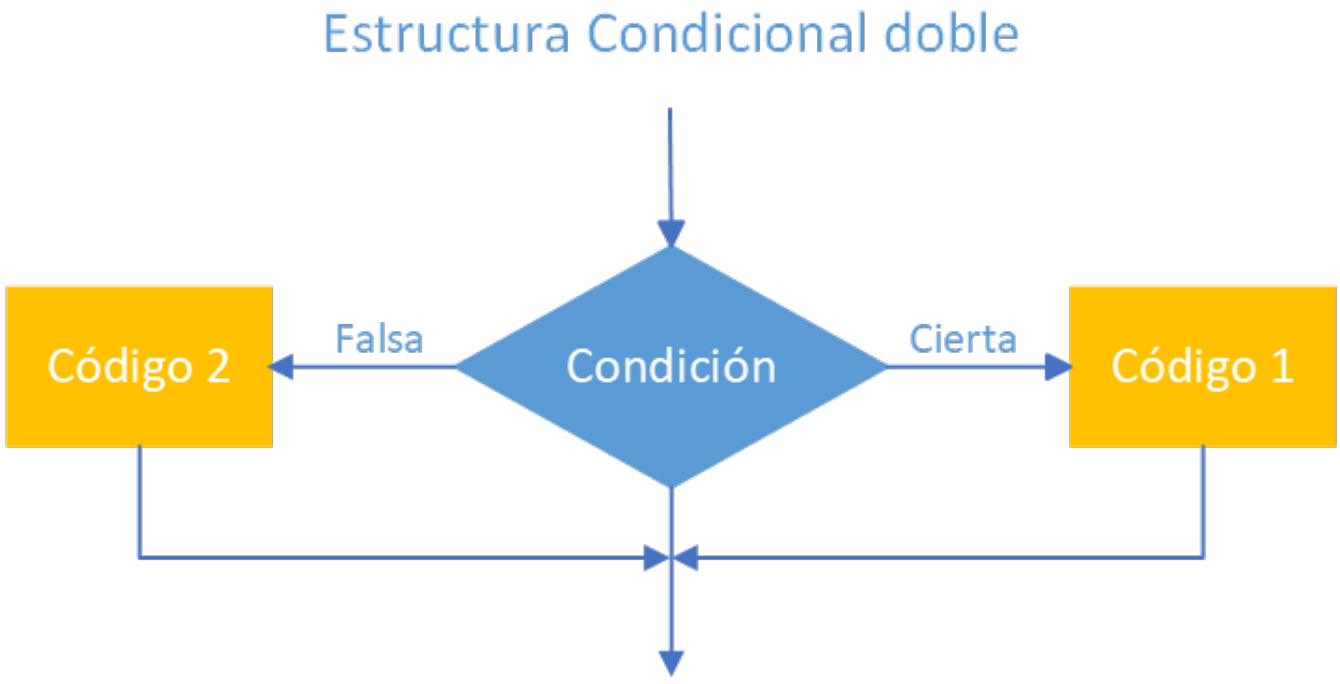
Versión de Python

La sentencia `match` está disponible a partir de **Python 3.10**.

8. Anexo enlaces y figuras

https://www.w3schools.com/python/python_conditions.asp





2.2.2 Introducción a los bucles en Python

Los **bucles** permiten repetir un conjunto de instrucciones varias veces.

En Python utilizaremos principalmente dos tipos:

- `for`, cuando queremos recorrer una secuencia o repetir algo un número conocido de veces.
- `while`, cuando queremos repetir mientras se cumpla una condición.

En estos ejemplos también veremos:

- `range()`
- acumuladores
- `break`
- recorrido de cadenas de texto

1. Código completo

```
# bucle for básico

# con índices y uso range
for i in range(5):
    print(i)

for i in range(10, 20):
    print(f"{i} ", end=" ")

# Suma números
suma = 0  # acumulador
ini = int(input("inicio: "))
fin = int(input("fin: "))

for i in range(ini, fin+1):
    print(i)
    suma += i  # equivale a suma = suma + i

print("\nSuma:", suma)  # \n salto de línea

# Bucle while

numero = int(input("Introduce un número (0 para terminar): "))
while numero != 0:
    print("Has introducido:", numero)
    numero = int(input("Introduce otro número (0 para terminar): "))
print("Fin del programa")

while True:  # el uso del break debe ser "cuidadoso..."
    numero = int(input("Introduce un número (0 para terminar): "))
    if numero == 0:
        break
    print("Has introducido:", numero)
print("Fin del programa")

# Se muestra ahora el uso de cadenas con bucle for

nombre = "www.aulas.local"
# a) Recorrer cada caracter de un str
for x in nombre:
    print(x)

# b) Recorrer cada caracter de un str
# usando el índice o posición si es necesario en el bucle
for i in range(len(nombre)):
    print(f"Pos{i}: {nombre[i]}")
```

2. Bucle `for` básico

Un bucle `for` permite repetir instrucciones recorriendo una secuencia de valores.

El primer ejemplo es:

```
for i in range(5):
    print(i)
```

`range(5)` genera los valores:

```
0
1
2
3
4
```

En cada vuelta del bucle, la variable `i` toma uno de esos valores.

Por tanto, la salida será:

```
0
1
2
3
4
```

> **El valor final no se incluye**
>
> `range(5)` genera valores desde `0` hasta `4`.
>
> El `5` no forma parte del recorrido.

3. Indicar inicio y fin con `range()`

También podemos indicar dónde empieza y dónde termina el recorrido:

```
for i in range(10, 20):
    print(f"{i} ", end=" ")
```

En este caso, `range(10, 20)` genera:

```
10 11 12 13 14 15 16 17 18 19
```

De nuevo, el límite final no se incluye.

3.1. EL PARÁMETRO `end` DE `print()`

Normalmente, `print()` termina con un salto de línea.

Por ejemplo:

```
print(10)
print(11)
```

produce:

```
10
11
```

Sin embargo:

```
print(f"{i} ", end=" ")
```

hace que cada valor se muestre en la misma línea.

El argumento `end` indica qué debe escribirse después de cada `print()`.

4. Utilizar un acumulador

El siguiente ejemplo suma todos los números comprendidos entre dos valores introducidos por el usuario:

```
suma = 0
ini = int(input("inicio: "))
fin = int(input("fin: "))
```

```
for i in range(ini, fin + 1):
    print(i)
    suma = suma + i

print("\nSuma:", suma)
```

La variable:

```
suma = 0
```

se utiliza como **acumulador**.

Un acumulador es una variable que va guardando un resultado parcial y se actualiza en cada vuelta del bucle.

En este caso:

```
suma = suma + i
```

significa:

1. tomar el valor actual de `suma`;
2. añadirle el valor de `i`;
3. guardar el nuevo resultado otra vez en `suma`.

4.1. EJEMPLO

Si el usuario introduce:

```
inicio: 1
fin: 4
```

el bucle recorrerá:

```
1
2
3
4
```

y el acumulador evolucionará así:

```
suma = 0
suma = 0 + 1
suma = 1 + 2
suma = 3 + 3
suma = 6 + 4
```

El resultado final será:

```
10
```

4.2. ¿POR QUÉ APARECE `fin + 1`?

El código utiliza:

```
range(ini, fin + 1)
```

porque `range()` no incluye el límite final.

Si queremos incluir el valor almacenado en `fin`, debemos utilizar `fin + 1`.

Forma abreviada

La instrucción:

```
suma = suma + i
```

también puede escribirse:

```
suma += i
```

Ambas hacen lo mismo.

5. Bucle `while`

Un bucle `while` repite instrucciones **mientras una condición sea verdadera**.

El ejemplo es:

```
numero = int(input("Introduce un número (0 para terminar): "))

while numero != 0:
    print("Has introducido:", numero)
    numero = int(input("Introduce otro número (0 para terminar): "))

print("Fin del programa")
```

La condición es:

```
numero != 0
```

El bucle continuará mientras el usuario introduzca un valor distinto de cero.

Cuando introduzca:

```
0
```

la condición pasará a ser falsa y el programa saldrá del bucle.

Después se ejecutará:

```
print("Fin del programa")
```

6. Actualizar la condición del `while`

Hay un detalle especialmente importante:

```
numero = int(input("Introduce otro número (0 para terminar): "))
```

Esta instrucción aparece **dentro del bucle**.

Gracias a ella, `numero` recibe un nuevo valor en cada iteración y la condición:

```
numero != 0
```

puede llegar a ser falsa.

> **Cuidado con los bucles infinitos**
>
> Si olvidásemos actualizar `numero`, la condición podría permanecer siempre verdadera y el bucle no terminaría.
>
> Por ejemplo:
>
> ```
> numero = 5
>
> while numero != 0:
>     print(numero)
> ```
>
> Este código imprimiría `5` continuamente.

7. `while True` y `break`

El siguiente ejemplo realiza prácticamente la misma tarea de otra forma:

```
while True:
    numero = int(input("Introduce un número (0 para terminar): "))

    if numero == 0:
        break

    print("Has introducido:", numero)

print("Fin del programa")
```

La condición:

```
while True:
```

es siempre verdadera.

Por tanto, en principio, este bucle no terminaría nunca.

Sin embargo, dentro aparece:

```
if numero == 0:
    break
```

La instrucción `break` provoca la **salida inmediata del bucle**.

Cuando el usuario introduce `0`, el programa abandona el `while` y continúa después de él.

7.1. DOS FORMAS DE RESOLVER EL MISMO PROBLEMA

Primera forma:

```
while numero != 0:
```

Segunda forma:

```
while True:
    ...
    if numero == 0:
        break
```

Ambas pueden ser válidas.

Para ejemplos sencillos, la primera suele expresar de forma muy clara la condición de repetición.

La segunda puede resultar útil cuando la condición para terminar aparece de forma natural dentro del bucle.

> ⚠ **Usar `break` con criterio**
>
> `break` no es incorrecto, pero conviene utilizarlo cuando realmente mejora la claridad del programa.
>
> Un exceso de salidas repartidas por un bucle puede hacer que el código sea más difícil de seguir.

8. Recorrer una cadena con `for`

Una cadena de texto puede recorrerse carácter a carácter.

Por ejemplo:

```
nombre = "www.aulas.local"

for x in nombre:
    print(x)
```

En cada iteración, `x` contiene uno de los caracteres de la cadena.

La salida comenzaría así:

```
w
w
w
.
a
u
...
```

Esta es normalmente la forma más sencilla de recorrer una cadena cuando solo necesitamos sus caracteres.

9. Recorrer una cadena utilizando índices

También podemos recorrer las posiciones de la cadena:

```
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")
```

Aquí se combinan varias ideas.

9.1. `len(nombre)`

Obtiene la longitud de la cadena.

Si:

```
nombre = "www.aulas.local"
```

`len(nombre)` devuelve el número de caracteres que contiene.

9.2. `range(len(nombre))`

Genera todas las posiciones válidas:

```
0
1
2
3
...
```

9.3. `nombre[i]`

Permite obtener el carácter situado en la posición `i` .

Por tanto, el programa mostrará algo similar a:

```
0: w
1: w
2: w
```

```
3: .
4: a
...
```

10. ¿Cuándo usar cada forma?

Si solo queremos los caracteres:

```
for caracter in nombre:
    print(caracter)
```

suele ser la opción más sencilla.

Si necesitamos conocer también la posición de cada carácter:

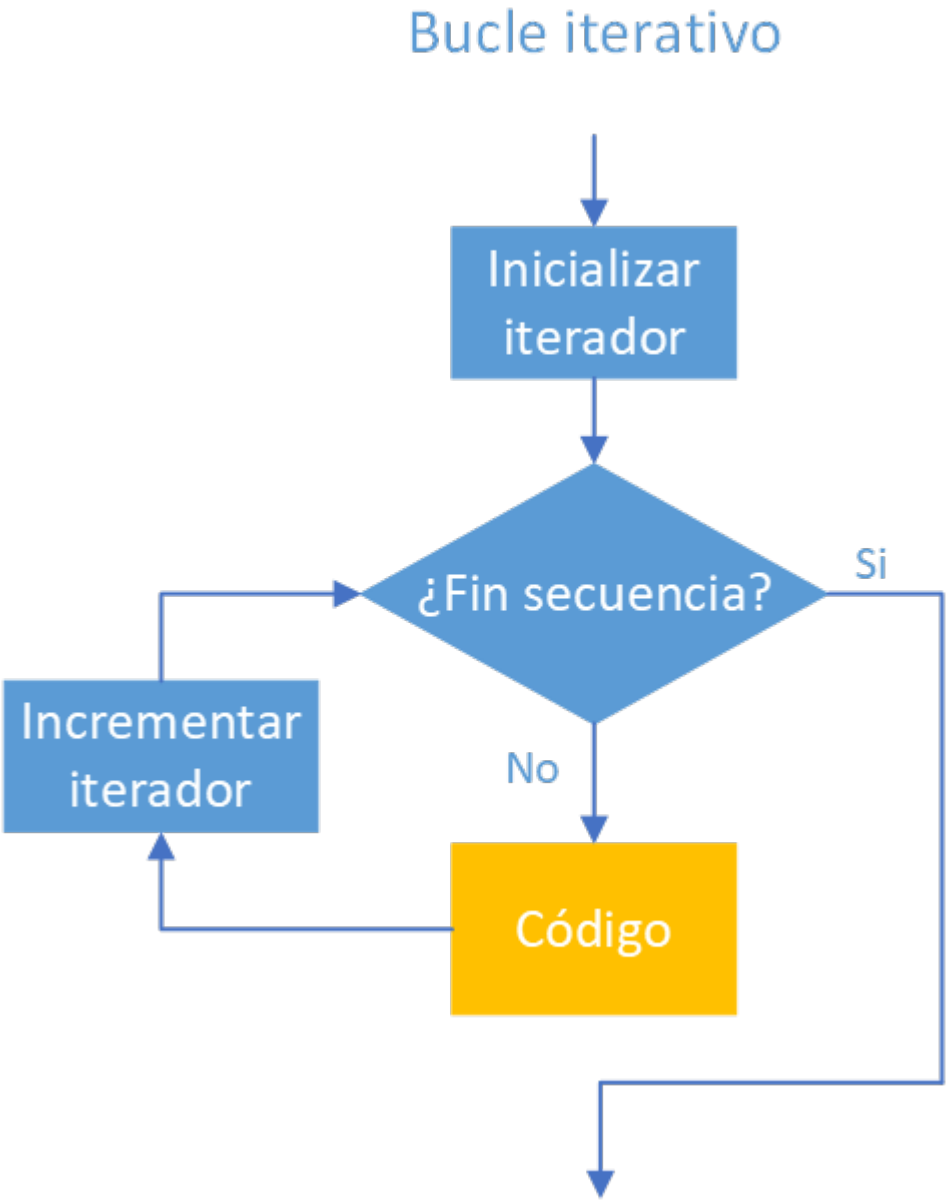
```
for i in range(len(nombre)):
    print(i, nombre[i])
```

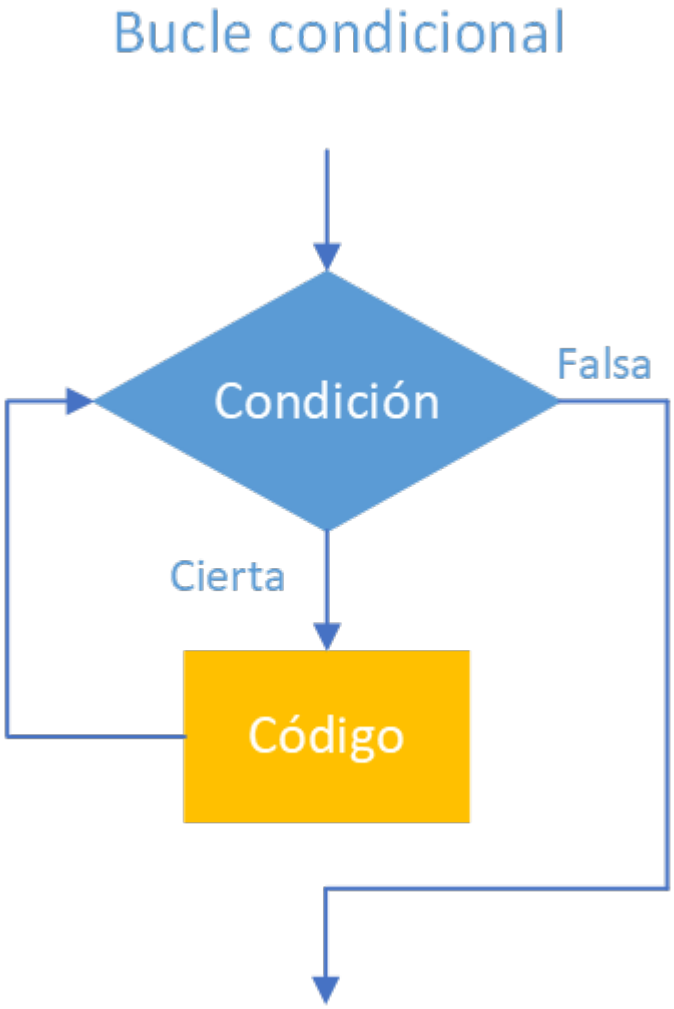
podemos recorrer los índices.

Más adelante veremos que Python también ofrece otras formas cómodas de obtener simultáneamente posición y valor.

11. Enlaces y figuras

- https://www.w3schools.com/python/python_for_loops.asp
- https://www.w3schools.com/python/python_while_loops.asp


Bucle iterativo
Inicializar iterador
¿Fin secuencia?
Si
No
Incrementar iterador
Código


Bucle condicional
Condición
Falsa
Cierta
Código

2.3 Estructuras Datos

2.3.1 Listas

Las **listas** permiten almacenar varios valores dentro de una misma variable.

Por ejemplo:

```
puertos = [22, 80, 443, 8080]
```

Una lista puede contener números, cadenas u otros tipos de datos. En estos ejemplos veremos cómo crear listas, acceder a sus elementos, obtener su longitud, extraer partes mediante *slicing*, modificar elementos, comprobar si una lista está vacía, recorrer listas, buscar elementos, ordenarlas y convertir cadenas en listas.

También aparecen dos contenidos algo más avanzados: las **comprensiones de listas** y el uso de `try/except`.

1. Crear listas y acceder a sus elementos

1.1. CÓDIGO COMPLETO

```python
# Definir lista y uso básico

# Lista de enteros
puertos = [22, 80, 443, 8080]

# mostar la lista y su tipo
print(puertos)
print(type(puertos))

# mostrar un elemento de la lista
print(puertos[0])
print(puertos[1])

# mostrar el tamaño (longitud => len())
print("Nº de elementos de la lista:", len(puertos))


# Ejemplos uso de lista: mostrar un rango
frutas = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print("frutas", frutas)
print("frutas[1:4]", frutas[1:4])

# Ejemplos uso de lista: generar nueva lista a partir de un rango
nueva_lista = frutas[1:4]
print("nueva_lista", nueva_lista)

# imprimir el último de una lista: usando len o con índice negativo...
print("frutas[len(frutas)-1]", frutas[len(frutas)-1])
print("Elemento finl: ", frutas[-1])

# modificar un elemento de la lista
print("Elemento 0: ", frutas[0])
print('ejecutamos frutas[0]="watermelon"')
frutas[0]="watermelon"
print("Elemento 0: ", frutas[0])


# Verificar lista vacía
lista = [1,2]
print("lista: ",lista)
print("len(lista):", len(lista))

# B) usando longitud
if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")

# A) preferible usando la lista como condición lógica
lista =  []
print("lista: ",lista)
print("len(lista):", len(lista))
if lista: # if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")


# Ejemplos formas de recorrer lista
lista_IP = ["192.168.1.1", "10.10.10.10", "172.16.100.100", "195.172.1.1"]

# los elementos con for
for x in lista_IP:
    print("IP: ", x)

# los elementos y sus índices con for
for i in range(len(lista_IP)):
    print(f"IP[{i}]: {lista_IP[i]}")

# Avanzado: comprensión de listas: [expresión for elemento in iterable]
[print(" -", x) for x in lista_IP ]
```

1.2. CREAR UNA LISTA

Una lista se escribe entre corchetes:

```python
puertos = [22, 80, 443, 8080]
```

En este caso contiene cuatro números enteros. Podemos mostrarla completa con `print(puertos)` y comprobar su tipo con `type(puertos)`.

1.3. ACCEDER A ELEMENTOS MEDIANTE SU POSICIÓN

Los elementos se numeran empezando en `0`:

```
print(puertos[0])
print(puertos[1])
```

> **Los índices comienzan en 0**
>
> El primer elemento de una lista está en la posición `0`, no en la `1`.

1.4. OBTENER LA LONGITUD CON `len()`

```
len(puertos)
```

devuelve el número de elementos de la lista.

1.5. OBTENER UNA PARTE DE UNA LISTA

Podemos utilizar *slicing*:

```
  frutas[1:4]
```

La posición inicial se incluye y la final no. También podemos guardar el fragmento obtenido en una nueva lista:

```
nueva_lista =   frutas[1:4]
```

1.6. ACCEDER AL ÚLTIMO ELEMENTO

Podemos calcular su posición:

```
  frutas[len(  frutas) - 1]
```

pero normalmente es más sencillo utilizar un índice negativo:

```
  frutas[-1]
```

Los índices negativos cuentan desde el final: `-1` es el último elemento, `-2` el penúltimo, etc.

1.7. MODIFICAR ELEMENTOS

Las listas son **mutables**, por lo que podemos modificar sus elementos directamente:

```
  frutas[0] = "watermelon"
```

> **Listas y cadenas no se comportan igual**
>
> Las cadenas son inmutables, pero las listas sí permiten modificar sus elementos.

2. Comprobar si una lista está vacía

Podemos hacerlo comprobando su longitud:

```
if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")
```

Python permite una forma más sencilla:

```
if lista:
    print("La lista contiene elementos")
```

```
else:
    print("Lista vacía")
```

Una lista vacía (`[]`) se considera falsa en una condición; una lista con elementos se considera verdadera.

3. Recorrer una lista

Si solo necesitamos los valores:

```
for x in lista_IP:
    print("IP: ", x)
```

Si necesitamos también las posiciones:

```
for i in range(len(lista_IP)):
    print(f"IP[{i}]: {lista_IP[i]}")
```

3.1. COMPRENSIÓN DE LISTAS

El script incluye:

```
[print(" -", x) for x in lista_IP]
```

La sintaxis general de una comprensión es:

```
[expresion for elemento in iterable]
```

> **Contenido avanzado**
>
> Aunque esta construcción funciona, las comprensiones de listas se utilizan sobre todo para **crear nuevas listas**, no para ejecutar acciones como `print()`.
>
> Para mostrar elementos es más claro:
>
> ```
> for x in lista_IP:
>     print(" -", x)
> ```

4. Buscar elementos en una lista

4.1. CÓDIGO COMPLETO

```
# BSUCAR elemento en la lista

# Con in => True/False

lista_IP = ["192.168.1.1", "10.10.10.10", "172.16.100.100", "195.172.1.1"]
print("lista_IP", lista_IP)

ip = input("IP a buscar: ")
if ip in lista_IP:
    print(f"Yes, {ip} is in {lista_IP}")
else:
    print(f"No, {ip} is not in {lista_IP}")

# Buscar con index si necesitamos la posición
# Nota: index genera excepción si no está

print("lista_IP", lista_IP)
ip = input("IP a buscar: ")

# pos = lista_IP.index(ip)  # Genera excepción si no está en la lista

# Para corregir excepción si no se encuentra
# a) usar in o count antes de index
if ip in lista_IP:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
else:
    print(f"{ip} NO está en {lista_IP}")

# b) se añade bloque try/except  para evitarlo
try:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
except ValueError:
    print(f"{ip} NO está en {lista_IP}")
```

4.2. BUSCAR CON `in`

La forma más sencilla de comprobar si un elemento existe es:

```
if ip in lista_IP:
```

`in` produce `True` si el elemento está en la lista y `False` si no está.

4.3. OBTENER LA POSICIÓN CON `index()`

```
lista_IP.index(ip)
```

devuelve la posición del elemento.

Si el elemento no existe, Python genera una excepción `ValueError`. Por eso en el ejemplo se comprueba primero con `in`.

4.4. ALTERNATIVA CON `try/except`

```
try:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
except ValueError:
    print(f"{ip} NO está en {lista_IP}")
```

Contenido algo más avanzado

`try/except` sirve para controlar excepciones durante la ejecución. Para una primera aproximación a listas, comprobar antes con `in` resulta más sencillo.

5. Ordenar y modificar listas

5.1. CÓDIGO COMPLETO

```
# ordenar listas: sort ,reverse
edades = [19, 17, 23, 56, 48, 100, 31]
print("edades:", edades)
```

```
edades.sort()
print("edades tras edades.sort():", edades)
edades.reverse()
print("edades tras edades.reverse():", edades)

# Modificar la lista: añadir al final
print("edades:", edades)
edades.append(20)
print("edades tras edades.append(20):", edades)

# Modificar la lista: añadir en cualquier posición:
edades.insert(1, 44)
print("edades tras edades.insert(1,44):", edades)

# Borrar un elemento
eliminado = edades.pop()
print("edades tras eliminado = edades.pop()", edades)
print("eliminado:", eliminado )

eliminado = edades.pop(0)
print("edades tras eliminado = edades.pop(0)", edades)
print("eliminado:", eliminado )

# Vaciar la lista
edades.clear()
print("edades tras clear", edades)

## CONVERTIR string en lista: split
users = "ana:luis:alfredo"
lista = users.split(":")
print("users:", users)
print("type(users)", type(users))
print("lista: ", lista)
print("type(lista)", type(lista))

# Ejemplo lectura notas con input y conversión a lista para calcular media
lectura = input("Notas separadas por el caracter ';': " )
print(lectura, type(lectura)) # es un string
notas = lectura.split(";") # es una lista
print(notas, type(notas))

suma = 0 # Almacena la suma de las notas
for x in notas:
    suma = suma + float(x)
media = suma/len(notas)
print("Suma: ", suma)
print(f"Media: {media:.2f}")

# Avanzado: conversión a números de una lista de string
edades = ["14", "21", "17", "33", "45"]
print("type(edades[0]):", type(edades[0]))
# comprensión de listas
edades =  [int(x) for x in edades]
print("type(edades[0]):", type(edades[0]))
```

5.2. ORDENAR CON `sort()`

```
edades.sort()
```

ordena la lista de menor a mayor y modifica la lista original.

5.3. INVERTIR CON `reverse()`

```
edades.reverse()
```

invierte el orden actual.

> **Ordenar e invertir no es lo mismo**
>
> `reverse()` no ordena: solo invierte el orden existente.

5.4. AÑADIR CON `append()`

```
edades.append(20)
```

añade un elemento al final.

5.5. INSERTAR EN UNA POSICIÓN

```
edades.insert(1, 44)
```

inserta `44` en la posición `1`.

5.6. ELIMINAR CON `pop()`

```
eliminado = edades.pop()
```

elimina el último elemento y lo devuelve. También podemos indicar una posición:

```
eliminado = edades.pop(0)
```

5.7. VACIAR UNA LISTA CON `clear()`

```
edades.clear()
```

elimina todos sus elementos.

6. Convertir una cadena en lista con `split()`

Si tenemos:

```
users = "ana:luis:alfredo"
```

podemos dividir la cadena usando `:` como separador:

```
lista = users.split(":")
```

El resultado será:

```
["ana", "luis", "alfredo"]
```

7. Leer varios valores y calcular una media

Si el usuario introduce:

```
5;7.5;8;6
```

la instrucción:

```
notas = lectura.split(";")
```

produce una lista similar a:

```
["5", "7.5", "8", "6"]
```

> **Siguen siendo cadenas**
>
> `split()` crea una lista, pero sus elementos siguen siendo de tipo `str`.

Por eso el programa convierte cada elemento al sumar:

```
for x in notas:
    suma = suma + float(x)
```

Finalmente calcula:

```
media = suma / len(notas)
```

8. Convertir todos los elementos de una lista

Partimos de:

```
edades = ["14", "21", "17", "33", "45"]
```

Con una comprensión:

```
edades = [int(x) for x in edades]
```

se obtiene una nueva lista de enteros.

Contenido avanzado

Una forma equivalente con un `for` tradicional sería:

```
nueva_lista = []

for x in edades:
    nueva_lista.append(int(x))

edades = nueva_lista
```

9. Enlaces

https://www.w3schools.com/python/python_lists.asp

2.3.2 Introducción a Python: Diccionarios

Los **diccionarios** son estructuras de datos que permiten almacenar información utilizando pares de **clave y valor**.

Por ejemplo:

```
coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}
```

En este diccionario:

- `"marca"` es una clave y `"Seat"` es su valor;
- `"modelo"` es una clave y `"Ateca"` es su valor;
- `"color"` es una clave y `"blanco"` es su valor;
- `"kilometros"` es una clave y `125000` es su valor.

Los diccionarios resultan especialmente útiles cuando queremos agrupar varios datos relacionados y acceder a ellos mediante un nombre significativo.

1. Código completo

```python
# Dictionaries are used to store data values in key:value pairs.
# A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}

# muestra el diccionario completo
print("coche:", coche)

# Mostar el valor de una clave: 2 posibilidades
# a) muestra un elemento con notación []
print('coche["color"]:', coche["color"])
print('coche["kilometros"]:', coche["kilometros"])
# b) muestra un elemento con método get
print('coche.get("modelo"):', coche.get("modelo"))


# Recorrer los valores del diccionario y sus claves
print("Bucle for que recorre diccionario")
for clave in coche:
    print(" *", clave, "=>", coche[clave])

# Otra forma de recorrer: items()
# `items()` permite obtener directamente la clave y el valor
#  en cada iteración. Rn muchos casos resulta más cómodo y legible.
print("Bucle for que recorre diccionario con .items()")
for clave, valor in coche.items():
    print(" - ", clave, "=>", valor)

# Modificar dicccionario: dos posibilidades
# a) con notación []
coche["color"] = "gris"
# b) con métoddo update
coche.update({"kilometros": 99999})
print("coche tras modificaciones:", coche)

# Añadir nueva clave a un diccionario
coche["taller"] = "Alterio reparaciones"
print("coche tras añadir clave:", coche)

# uso de values() y keys()
valores = coche.values()
print("valores:", valores)
claves = coche.keys()
print("keys:", claves)

# las variables asignadas con values y keys()
#  se actualizan tras cambios en el diccionario
print('Se añade precio: coche["precio"] = 17000')
coche["precio"] = 17000
print("valores:", valores)
print("keys:", claves)
```

2. Crear un diccionario

Un diccionario se escribe utilizando llaves:

```python
coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}
```

Cada elemento tiene la forma:

```
clave: valor
```

Por ejemplo:

```
"marca": "Seat"
```

La clave es:

```
marca
```

y el valor asociado es:

```
Seat
```

Claves y valores

En una lista accedemos a los elementos normalmente mediante su posición:

```
lista[0]
```

En un diccionario utilizamos una **clave**:

```
coche["marca"]
```

Esto hace que el código pueda resultar más descriptivo.

3. Mostrar el diccionario completo

Podemos mostrar todo el diccionario utilizando `print()`:

```
print("coche:", coche)
```

Obtendremos un resultado similar a:

```
coche: {'marca': 'Seat', 'modelo': 'Ateca', 'color': 'blanco', 'kilometros': 125000}
```

4. Acceder a un valor mediante su clave

Podemos utilizar corchetes:

```
coche["color"]
```

Por ejemplo:

```
print(coche["color"])
```

mostrará:

```
blanco
```

También podemos consultar:

```
coche["kilometros"]
```

y obtendremos:

```
125000
```

La clave nos permite indicar directamente qué dato queremos consultar.

5. Acceder con `get()`

Otra posibilidad es utilizar el método:

```
get()
```

Por ejemplo:

```
coche.get("modelo")
```

devuelve:

```
Ateca
```

Por tanto:

```
print(coche.get("modelo"))
```

es otra forma de consultar un valor.

5.1. DIFERENCIA ENTRE `[]` Y `get()`

Mientras la clave exista, estas dos expresiones devuelven el mismo resultado:

```
coche["modelo"]
```

```
coche.get("modelo")
```

La diferencia aparece cuando la clave **no existe**.

Si intentamos:

```
coche["precio"]
```

antes de haber creado esa clave, Python genera:

```
KeyError
```

Sin embargo:

```
coche.get("precio")
```

devuelve por defecto:

```
None
```

sin detener el programa.

> **`get()` permite indicar un valor por defecto**
>
> Podemos escribir:
>
> ```
> coche.get("precio", 0)
> ```
>
> Si `"precio"` no existe, obtendremos `0`.

6. Recorrer un diccionario con `for`

El ejemplo utiliza:

```
for x in coche:
    print(" *", x, "=>", coche[x])
```

Cuando recorremos directamente un diccionario:

```
for x in coche:
```

la variable `x` va tomando las **claves** del diccionario.

Por ejemplo:

```
marca
modelo
```

```
color
kilometros
```

Podemos utilizar la clave para consultar su valor:

```
coche[x]
```

De esta forma podemos mostrar ambos:

```
marca => Seat
modelo => Ateca
color => blanco
kilometros => 125000
```

7. Modificar un valor

Los diccionarios son **mutables**, por lo que podemos cambiar sus valores.

Por ejemplo:

```
coche["color"] = "gris"
```

La clave `"color"` ya existe, por lo que su valor pasa de:

```
blanco
```

a:

```
gris
```

8. Modificar mediante `update()`

También podemos utilizar:

```
coche.update({"kilometros": 99999})
```

El método `update()` recibe otro diccionario con los datos que queremos actualizar.

Después de ejecutar esa instrucción, el valor asociado a `"kilometros"` será:

```
99999
```

Para modificar un único elemento, tanto:

```
coche["kilometros"] = 99999
```

como:

```
coche.update({"kilometros": 99999})
```

son posibles.

9. Añadir una nueva clave

La misma sintaxis que utilizamos para modificar puede servir para crear un elemento nuevo:

```
coche["taller"] = "Alterio reparaciones"
```

Como la clave `"taller"` no existía, Python la añade al diccionario.

Ahora tendremos también:

```
taller => Alterio reparaciones
```

Modificar o añadir

Esta instrucción:

```
diccionario[clave] = valor
```

hace dos cosas posibles:

- si la clave ya existe, modifica su valor;
- si no existe, crea una nueva entrada.

10. Obtener los valores con `values()`

El método:

```
coche.values()
```

permite obtener una vista de todos los valores del diccionario.

En el ejemplo:

```
valores = coche.values()
print("valores:", valores)
```

aparecerán los valores correspondientes a elementos como:

```
Seat
Ateca
gris
99999
Alterio reparaciones
```

11. Obtener las claves con `keys()`

El método:

```
coche.keys()
```

permite obtener las claves.

Por ejemplo:

```
keys = coche.keys()
print("keys:", keys)
```

incluirá claves como:

```
marca
modelo
color
kilometros
taller
```

12. Las vistas `keys()` y `values()` se actualizan

El ejemplo guarda:

```
valores = coche.values()
keys = coche.keys()
```

Después añade:

```
coche["precio"] = 17000
```

Y vuelve a mostrar:

```
print("valores:", valores)
print("keys:", keys)
```

La nueva clave y el nuevo valor también aparecen.

Esto sucede porque `keys()` y `values()` devuelven **vistas del diccionario**, no copias independientes.

> **Una vista refleja los cambios**
>
> Si el diccionario cambia, las vistas obtenidas con:
>
> ```
> keys()
> values()
> ```
>
> reflejan esos cambios.

13. Otra forma habitual de recorrer un diccionario

Además de:

```
for clave in coche:
    print(clave, coche[clave])
```

existe el método `items()` :

```
for clave, valor in coche.items():
    print(clave, "=>", valor)
```

`items()` permite obtener directamente la clave y el valor en cada iteración. Para muchos casos resulta especialmente cómodo y legible.

14. Enlaces

https://www.w3schools.com/python/python_dictionaries.asp

2.3.3 Estructuras de datos combinadas: listas y diccionarios

Hasta ahora ha trabajado con **listas** y **diccionarios** por separado. Sin embargo, en programas reales es muy habitual combinar ambas estructuras.

En este tema se presentan dos casos frecuentes:

1. un **diccionario que contiene una lista**;
2. una **lista cuyos elementos son diccionarios**.

1. Diccionario con una lista como valor

Un diccionario puede almacenar distintos tipos de valores. entre ellos una lista:

Por ejemplo:

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20",
    "servicios": ["ssh", "http", "https"]
}
```

- `"nombre"` contiene una cadena;
- `"ip"` contiene una cadena;
- `"servicios"` contiene una **lista**.

Esto resulta útil cuando un elemento puede tener varios valores relacionados. En este caso, un servidor puede tener varios servicios activos.

Uan vez definido el diccionario podría mostar sus dos primeros valores

```
print("Servidor:", servidor["nombre"])
print("IP:", servidor["ip"])
```

Y para el tercer valor se recorre la lista cuyo identificador es `servidor["servicios"]`

```
# print(servidor["servicios"])
print("Servicios:")
for servicio in servidor["servicios"]:
    print(" -", servicio)
```

Al ser una lista, para acceder a un element ose usa la notación de corchetes tras el nombre de la lista `servidor["servicios"][indice]`

```
# Imprime un servicio de la lista
print("Primer servicio:", servidor["servicios"][0])
```

Lectura de izquierda a derecha

Esta expresión:

```
servidor["servicios"][0]
```

puede interpretarse como:

1. busca la clave `"servicios"`;
2. obtiene la lista asociada;
3. accede al elemento situado en la posición `0`.

También puede utilizar los métodos habituales de las listas. En este caso se añade un servicio con `append`: No está creando una nueva clave sino modificando la lista:

```
servidor["servicios"].append("ftp")

print("Servicios tras añadir ftp:")
```

```
for servicio in servidor["servicios"]:
    print(" -", servicio)
```

2. Lista cuyos elementos son diccionarios

El segundo caso es muy habitual. Suponga que desea guardar información sobre varios servidores.

Puede crear una lista:

```
servidores = [
    {
        "nombre": "web01",
        "ip": "192.168.1.20"
    },
    {
        "nombre": "db01",
        "ip": "192.168.1.30"
    },
    {
        "nombre": "backup01",
        "ip": "192.168.1.40"
    }
]
```

En este caso tiene una **lista** con tres elementos. Y vada elemento es un **diccionario**.

Se puede acceder al primer elemento de la lista. Ese elemento es un diccionario.

```
print("Primer servidor:")
print(servidores[0])
```

El resultado será:

```
{
    "nombre": "web01",
    "ip": "192.168.1.20"
}
```

Al ser `servidores[0]` un diccionario se puede acceder a sus valores:

```
print("Nombre del primer servidor:")
print(servidores[0]["nombre"])
```

El resultado será:

```
web01
```

Y para recorrer la lista mostrando los valores

```
print("Lista de servidores:")
for servidor in servidores:
    print(servidor["nombre"], "=>", servidor["ip"])
```

3. Anexo. Recomendaciones

- Utilizar nombres de variables descriptivos. En estructuras combinadas es especialmente importante utilizar nombres claros. Por ejemplo:

  ```
  for servidor in servidores:
  ```

  es más fácil de entender que:

  ```
  for x in servidores:
  ```

- Separar los pasos POR CLARIDAD. En vez de:

  ```
  print(servidores[0]["nombre"])
  ```

  puede escribir:

  ```
  primer_servidor = servidores[0]
  print(primer_servidor["nombre"])
  ```

Ambas formas son correctas, pero la segunda puede resultar más legible al principio.

2.4 Ejercicios I

2.4.1 Ejercicios Tema 01

Ejercicio 1. Variables y tipos

Crea tres variables:

- `nombre`, con tu nombre.
- `edad`, con tu edad.
- `altura`, con tu altura en metros.

Muestra el contenido de las tres variables y, después, muestra el tipo de cada una utilizando `type()`.

Ejercicio 2. Cadenas

Partiendo de:

```
frase = "Python es divertido"
```

haz que el programa:

1. Muestre la frase en minúsculas.
2. Muestre su longitud.
3. Compruebe si contiene la palabra `"python"` sin distinguir mayúsculas y minúsculas.
4. Muestre la frase al revés.

Ejercicio 3. Booleanos

Crea una variable:

```
es_administrador = False
```

Utiliza un `if` y un `else` para mostrar:

```
Acceso de administrador
```

si su valor es verdadero, y:

```
Acceso de usuario
```

si es falso.

Después cambia su valor a `True` y observa el resultado.

Ejercicio 4. Conversión de tipos

Pide al usuario el precio de un producto y una cantidad:

```
Precio:
Cantidad:
```

Convierte los valores a los tipos adecuados y calcula el importe total:

```
importe = precio * cantidad
```

Muestra finalmente el resultado.

Ejercicio 5. Pequeño reto

Pide al usuario su nombre y su año de nacimiento. Calcula aproximadamente su edad suponiendo que estamos en 2026 y muestra un mensaje similar a:

```
Ana, tienes aproximadamente 20 años.
```

Pista

Recuerda que `input()` devuelve siempre una cadena. Para realizar la resta tendrás que convertir el año de nacimiento a un tipo numérico.

2.4.2 Ejercicios

Ejercicio 1. Mayor de edad

Pide al usuario su edad.

Si tiene 18 años o más, muestra:

```
Es mayor de edad
```

Si no, muestra:

```
Es menor de edad
```

Ejercicio 2. Número positivo, negativo o cero

Pide al usuario un número entero y determina si es:

- positivo;
- negativo;
- cero.

Utiliza `if`, `elif` y `else`.

Ejercicio 3. Acceso al sistema

Pide al usuario:

- su edad;
- si dispone de saldo.

Para simplificar, representa el saldo mediante un número.

Permite el acceso solamente si tiene al menos 18 años **y** el saldo es mayor que 0.

Ejercicio 4. Comparar dos números

Pide al usuario dos números enteros y muestra cuál es mayor.

Si son iguales, indícalo también.

Ejercicio 5. Menú con `match`

Muestra este menú:

```
1. Crear usuario
2. Eliminar usuario
3. Salir
```

Pide una opción y utiliza `match` para mostrar la acción seleccionada.

Para cualquier otro número, muestra:

```
Opción incorrecta
```

2.4.3 Ejercicios

Ejercicio 1. Contar del 0 al 9

Utiliza un bucle `for` y `range()` para mostrar los números del 0 al 9.

Ejercicio 2. Contar del 1 al 10

Modifica el ejercicio anterior para mostrar:

```
1
2
3
...
10
```

Ejercicio 3. Sumar del 1 a un número

Pide al usuario un número entero positivo.

Utiliza un acumulador para calcular la suma desde `1` hasta ese número.

Por ejemplo, si introduce `5`:

```
1 + 2 + 3 + 4 + 5 = 15
```

Ejercicio 4. Repetir hasta introducir cero

Pide números al usuario continuamente.

Cada vez que introduzca un número distinto de `0`, muéstralo.

Cuando escriba `0`, termina el programa.

Resuélvelo utilizando `while` **sin utilizar** `break`.

Ejercicio 5. El mismo problema con `break`

Repite el ejercicio anterior, pero esta vez utiliza:

```
while True:
```

y una instrucción `break`.

Ejercicio 6. Recorrer una palabra

Pide al usuario una palabra y muestra cada carácter en una línea distinta.

Por ejemplo:

```
Palabra: casa
c
a
s
a
```

Ejercicio 7. Mostrar posiciones y caracteres

Pide una palabra al usuario y muestra la posición y el carácter correspondiente.

Por ejemplo:

```
Palabra: sol
0: s
1: o
2: l
```

2.4.4 Ejercicios

Ejercicio 1. Crear y consultar una lista

Crea una lista con los puertos `22`, `53`, `80`, `443` y `8080`. Muestra la lista completa, el primer elemento, el último y el número de elementos.

Ejercicio 2. Modificar una lista

Partiendo de:

```
usuarios = ["ana", "luis", "marta"]
```

cambia `"ana"` por `"alfredo"`, añade `"raquel"` al final e inserta `"carlos"` en la posición `1`.

Ejercicio 3. Buscar una IP

Crea:

```
ips = ["192.168.1.1", "10.0.0.1", "172.16.0.1"]
```

Pide al usuario una IP y muestra si está o no en la lista utilizando `in`.

Ejercicio 4. Posición de una IP

Amplía el ejercicio anterior para mostrar también la posición cuando exista, evitando que se produzca `ValueError`.

Ejercicio 5. Ordenar edades

Crea:

```
edades = [34, 18, 55, 21, 40]
```

Ordénala de menor a mayor, muéstrala, inviértela y vuelve a mostrarla.

Ejercicio 6. Usuarios separados por comas

Pide nombres separados por comas, conviértelos en una lista con `split()` y muestra cada usuario en una línea.

Ejercicio 7. Calcular la media

Pide varias notas separadas por `;`, conviértelas a números y calcula la media.

Ejercicio 8. Conversión mediante comprensión

Partiendo de:

```
numeros = ["10", "20", "30", "40"]
```

crea una lista de enteros mediante una comprensión de listas.

2.4.5 Ejercicios

Ejercicio 1. Crear un diccionario

Crea un diccionario llamado `servidor` con estas claves:

- `"nombre"`
- `"ip"`
- `"puerto"`

Asigna valores de ejemplo y muestra el diccionario completo.

Ejercicio 2. Consultar elementos

Partiendo de:

```
usuario = {
    "nombre": "Ana",
    "edad": 21,
    "activo": True
}
```

muestra por separado:

1. el nombre;
2. la edad;
3. si está activo.

Ejercicio 3. Modificar y añadir información

Partiendo del diccionario anterior:

1. cambia la edad a `22`;
2. añade una clave `"email"`;
3. muestra el diccionario completo.

Ejercicio 4. Buscar una clave

Crea:

```
equipo = {
    "hostname": "server01",
    "ip": "192.168.1.10",
    "so": "Linux"
}
```

Pide al usuario el nombre de una clave.

Comprueba si existe utilizando:

```
in
```

y muestra su valor si existe.

Pista

Puedes comprobar una clave así:

```
if clave in equipo:
```

Ejercicio 5. Recorrer clave y valor

Recorre este diccionario:

```
puertos = {
    "ssh": 22,
    "http": 80,
    "https": 443
}
```

y muestra:

```
ssh => 22
http => 80
https => 443
```

Ejercicio 6. Utilizar `get()`

Partiendo de:

```
persona = {
    "nombre": "Luis",
    "ciudad": "Cádiz"
}
```

consulta con `get()`:

- `"nombre"`;
- `"telefono"`.

Para `"telefono"`, utiliza como valor por defecto:

```
No disponible
```

2.5 Módulos/funciones

2.5.1 Introducción a Python: uso módulos y funciones

En administración de sistemas, las funciones son bloques de código reutilizables diseñados para automatizar tareas repetitivas. Las funciones pueden ser propias o que ya vengan en módulos de la biblioteca estándar o de terceros

Por ejemplo, el módulo `os` de la biblioteca estándar de Python permite interactuar directamente con el sistema operativo (archivos, directorios, procesos y rutas) de forma independiente del SO.

1. Usando un módulo

Para invocar (ejecutar) una función que pertenece a un módulo en Python, se sigue esta sintaxis:

1. **Importar el módulo:** Declarar la dependencia al inicio del script (`import modulo`).
2. **Notación de punto y paréntesis:** Conectar el nombre del módulo con la función (`modulo.funcion()`).
3. **Argumentos entre paréntesis `()` :** Puede no tener argumentos, o que sean obligatorios u opcionales, puede necesitar uno o varios, etc.

2. Función SIN parámetros

Una función sin parámetros no requiere datos de entrada para realizar su trabajo; obtiene la información directamente del entorno de ejecución. Es el caso de `os.getcwd()` :

- **Propósito:** Devuelve la ruta absoluta del directorio de trabajo actual (*Get Current Working Directory*).
- **Equivalente CLI:** Comando `pwd` en Linux/macOS o `cd` en Windows CMD.

```
import os

# Se invoca dejando los paréntesis vacíos
directorio_actual = os.getcwd()

print(f"El script se está ejecutando desde: {directorio_actual}")
```

3. Función CON parámetro obligatorio

Una función con parámetros requiere que le pases uno o más datos (argumentos) dentro de los paréntesis para poder operar. Es el caso de `os.mkdir(path)` :

- **Propósito:** Crea un nuevo directorio en el sistema de archivos en la ruta indicada.
- **Equivalente CLI:** Comando `mkdir <nombre_directorio>` .

```
import os

# Definimos el argumento: el nombre de la carpeta que queremos crear
carpeta_backup = "/var/backups/diarios"

# Invocación pasando el parámetro (obligatorio)
os.mkdir(carpeta_backup)
```

```
print(f"Directorio creado correctamente en: {carpeta_backup}")
```

> Nota para administradores: Si la carpeta ya existe o no se tienen permisos de escritura en la ruta destino, Python interrumpirá el script con una excepción. A diferencia de los scripts de Bash o las llamadas en C (donde las funciones devuelven un código de salida como 0 o 1), os.mkdir() gestiona los resultados mediante el flujo de excepciones:
>
> - Si la carpeta se crea correctamente: La función finaliza en silencio devolviendo None.
> - Si ocurre un error: No devuelve un código de error, sino que interrumpe la ejecución lanzando una excepción:
>   - FileExistsError: La carpeta ya existe en esa ruta.
>   - PermissionError: El usuario que ejecuta el script no tiene permisos de escritura.
>   - FileNotFoundError: La ruta intermedia especificada no existe (por ejemplo, intentar crear a/b/c cuando a o b no existen).

4. Función CON parámetro opcional

El método `os.listdir(path)` retorna:

- con parametro `path` el contenido del directorio indicado en `path`
- sin parámetro `path` el contenido del directorio actual

```
# Método con argumento opcional
files_dir_actual = os.listdir()
files_dir_backup = os.listdir(carpeta_backup)
print(files_dir_actual)
print(files_dir_backup)
```

5. Función con más de un parámetro

Un ejemplo con dos parámetros (obligatorios) es la función `os.rename(src, dst)` que sirve para renombrar o mover un archivo o directorio de una ubicación a otra. Es equivalente al comando `mv` en Linux:

1. Primer parámetro (`src` / *source*): La ruta del archivo o carpeta original que ya existe.
2. Segundo parámetro (`dst` / *destination*): La ruta o nombre final del archivo o carpeta.

```
import os

# 1. Definimos los datos que necesita la función
log_actual = "/var/log/servidor.log"
log_rotado = "/var/log/servidor.log.old"

# 2. Invocamos la función pasando ambos parámetros separados por una coma
#    El orden es importante
os.rename(log_actual, log_rotado)

print("El archivo se ha renombrado correctamente.")
```

> Al igual que `os.mkdir()`, devuelve `None` y gestiona los fallos mediante excepciones (`FileNotFoundError`, `PermissionError`, etc.).

6. Argumentos: Posicionales vs con nombre

En Python cuando una función recibe varios parámetros se pueden pasar de dos formas. Usemos como ejemplo `os.rename(src, dst)`:

1. Por posición (lo más habitual):El orden de los argumentos dentro del paréntesis determina a qué parámetro corresponde cada uno (el primer valor va a `src` y el segundo a `dst`).

```
os.rename("/tmp/archivo.txt", "/tmp/archivo_nuevo.txt")
```

2. Por nombre de parámetro (*keyword arguments*): Puede especificar explícitamente el nombre del parámetro. En este caso, el orden en el que los escriba no importa.

```
os.rename(dst="/tmp/archivo_nuevo.txt", src="/tmp/archivo.txt")
```

7. Ejemplo completo

```
import os

# Método sin argumentos:
# Se invoca dejando los paréntesis vacíos
directorio_actual = os.getcwd()
print(f"El script se está ejecutando desde: {directorio_actual}")

# Método con argumento obligatorio:
# Definimos el argumento: el nombre de la carpeta que queremos crear
carpeta_backup = "bk"

# Invocación pasando el parámetro (obligatorio)
os.mkdir(carpeta_backup)
print(f"Directorio creado correctamente en: {carpeta_backup}")

# Método con argumento opcional
files_dir_actual = os.listdir()
files_dir_backup = os.listdir(carpeta_backup)
print(files_dir_actual)
print(files_dir_backup)


## Método con 2 argumentos posicionales
# 1. Definimos los datos que necesita la función
log_actual = "servidor.log"
log_rotado = "servidor.log.old"

# 1.1 Crear archivo para que rename no falle,,,
with open(log_actual, mode="a", encoding="utf-8") as archivo:
    pass

# 2. Invocamos la función pasando ambos parámetros separados por una coma
os.rename(log_actual, log_rotado)

print("El archivo se ha renombrado correctamente.")


# Método con argumentos con nombre
os.rename(dst="/tmp/archivo_nuevo.txt", src="/tmp/archivo.txt")
```

2.5.2 Introducción a Python: Funciones II

Las funciones son bloques de código reutilizables diseñados para automatizar tareas repetitivas.

- Para definir una función en Python, se utiliza la palabra reservada `def`, seguida del nombre de la función, paréntesis `()` y dos puntos `:`. Todo el código que pertenezca a la función debe estar indentado (desplazado a la derecha).
- Para ejecutar una función hay que invocarla por su nombre con los paréntesis `()`.

1. Funciones SIN parámetros

Una función sin parámetros no requiere datos de entrada para realizar su trabajo.

```
# Función sin parámetros (siempre hace lo mismo)
def imprime_separador_simple():
    print("-"*25)

# Invocación ejecuta la función
imprime_separador_simple()
imprime_separador_simple()
```

2. Funciones CON un parámetro

Una función con parámetro requiere que le pase un parámetro o argumento dentro de los paréntesis para poder operar de forma dinámica.

```
# Función con un parámetro
def imprime_separador(longitud):
    print("-"*longitud)

# Invocaciones con diferentes expresiones
imprime_separador(5)
imprime_separador(10)
tam = 15
imprime_separador(tam)
imprime_separador(tam*2)
```

3. Funciones CON múltiples parámetros

Puedes definir tantos parámetros como necesites, separándolos por comas. Cuando una función recibe varios parámetros, los datos se pueden enviar en la invocación de dos formas distintas:

```
# Función con dos parámetros
def imprime_linea(letra, longitud):
    print(letra*longitud)

# Invocaciones posicionales con diferentes valores
imprime_linea("*", 5)
imprime_linea("#", tam)

# Invocación con nombres, puede cambiar el orden
imprime_linea(longitud=10, letra="&")
```

4. Función que usa otra funcion

```
# Utiliza en la definición otra función
def imprime_cabecera(titulo):
    imprime_linea("+",10)
    print(titulo)
    imprime_linea("-",10)

imprime_cabecera("Latencias")
```

5. Funciones que retornan un resultado (return)

En Python las funciones por defecto no devuelven ningún valor explícito, retornando `None`. Sin embargo, si necesita que la función devuelva un cálculo final al script principal se utiliza la palabra clave `return`.

```
# función que DEVUELVE COMO RESULTADO la suma de valores de una lista
def sumaLista(lista):
```

```
    suma = 0
    for x in lista:
        suma += x   # suma = suma + x
    return suma

ventas = [1,2,3,4,5]
print("suma: ", sumaLista(ventas))

ventasTotales = sumaLista(ventas)
print("ventas:", ventas)
print("ventasTotales:", ventasTotales)
```

```
# función que DEVUELVE COMO RESULTADO la media USANDO LA FUNCIÓN sumaLista
def media(lista):
    media = sumaLista(lista)/len(lista)
    return media

edades = [23, 46, 56, 72, 12, 34]
print("edades:", edades)
print("sumaLista(edades):", sumaLista(edades))
print("media(edades):", media(edades))
```

Valores por defecto

Se pueden definir valores por defecto para los argumentos

```
# ejemplo valores por defecto
def imprime_test(longitud=20, letra="-"):
    print(letra*longitud)

imprime_test()
imprime_test(10)
imprime_test(15, "*")
```

Tenga en cuenta que en las llamadas a las funciones el orden de los parámetros debe coincidir con el orden de los argumentos en la función:

```
# Parametro posicionales
def carga_recibo(saldo, recibo):
    if recibo < 0:
        print("recibo debe ser > 0")
    elif saldo < recibo:
        print("saldo inferior a recibo")
    else:
        saldo = saldo - recibo

    return saldo
```

Y un ejemplo de ejecución correcta e incorrecta:

```
saldo_actual = 100
recibo_luz = 45
recibo_agua = 40
print("Saldo: ", saldo_actual)

# El orden importa, sin nombres la correspondencia es posicional
saldo_actual = carga_recibo(saldo_actual, recibo_luz)  # Correcto
print("Saldo tras recibo: ", saldo_actual)
print("llamada incorrecta que además modifica el saldo !!!!")
saldo_actual = carga_recibo(recibo_agua, saldo_actual)  # incorrecto !!!!
print("Saldo tras recibo: ", saldo_actual)
```

Y un ejemplo con uso de nombres:

```
# otra opción: con nombres
saldo_actual = carga_recibo(recibo=recibo_luz, saldo=saldo_actual) # con nombres
```

6. Anexo. Uso de Módulos

Otro concepto interesantes es el de los módulos en Python:

> Modules are files that contain Python code, which allow us to divide code into multiple files so that it is more organized and easier to reuse code.
>
> A module in Python is simply a file with a .py extension that contains Python code (within this file, we can include function definitions, variables, classes, etc.).

Tiene una explicación detallada con ejemplos en este enlace: https://www.luisllamas.es/en/what-are-python-modules/

Un caso práctico a partir de los ejemplos previos: se agrupan las funciones `sumaLista`, `media`, etc, en un fichero llamado `utiles.py`.

utiles.py

```
# DEVUELVE COMO RESULTADO la suma de valores de una lista
def sumaLista(lista):
    suma = 0
    for x in lista:
        suma += x   # suma = suma + x
    return suma

# DEVUELVE COMO RESULTADO la media USANDO LA FUNCIÓN sumaLista
def media(lista):
    media = sumaLista(lista)/len(lista)
    return media
```

Y otro fichero con la aplicación:

main.py

```
# El nombre del módulo a importar es el nombre del fichero
# Se importan las funciones que use del móudlo
from utiles import sumaLista, media

ventas = [1,2,3,4,5]
ventasTotales = sumaLista(ventas)

print("Lista ventas:", ventas)
print("Suma Ventas:", ventasTotales)
```

Y la ejecuión y su salida

```
$ python3 main.py
Lista ventas: [1, 2, 3, 4, 5]
Suma Ventas: 15
$
```

2.6 Introducción a Python - Excepciones

2.6.1 1. Introducción

Cuando un programa encuentra un problema durante su ejecución, Python puede generar una **excepción**. Ya vimos un ejemplo al utilizar `os.mkdir()` para crear directorios. Por ejemplo:

```
import os

os.mkdir("/root/nueva_carpeta")
```

Si el usuario que ejecuta el programa no tiene permisos para crear el directorio, Python interrumpe la ejecución y genera una excepción. Una posible salida sería:

```
Traceback (most recent call last):
  File "13_excepciones.py", line 25, in <module>
    os.mkdir("/root/nueva_carpeta")
PermissionError: [Errno 13] Permission denied: '/root/nueva_carpeta'
```

En este caso, la excepción es:

```
PermissionError
```

2.6.2 2. ¿Qué es una excepción?

Una excepción representa una situación anómala que ocurre mientras el programa se está ejecutando. Algunos ejemplos habituales son:

- intentar convertir a número un texto que no contiene un número;
- dividir entre cero;
- acceder a una posición que no existe en una lista;
- buscar una clave inexistente en un diccionario;
- abrir un fichero que no existe;
- crear un directorio sin permisos suficientes.

Si no tratamos la excepción, el programa normalmente se detiene y Python muestra un **traceback**.

> **Excepción no significa necesariamente error de programación**
>
> Una excepción también puede representar situaciones perfectamente posibles durante la ejecución.
>
> Por ejemplo, que un usuario escriba un dato incorrecto o que un fichero no exista.

2.6.3 3. Excepciones con `os.mkdir()`

La función `os.mkdir(ruta)` no devuelve un código de error cuando algo falla. Si el directorio se crea correctamente, la función termina y devuelve:

```
None
```

Si se produce un problema, genera una excepción. Algunas excepciones habituales son:

- `FileExistsError`: Se produce si el directorio ya existe.
- `PermissionError`: Se produce si no tenemos permisos suficientes.
- `FileNotFoundError`: Puede aparecer si algún directorio intermedio no existe.

2.6.4 4. La estructura `try ... except`

Para evitar que el programa se detenga automáticamente podemos **capturar** la excepción.La estructura básica es:

```
try:
    # código que puede generar una excepción
except TipoDeExcepcion:
    # qué hacer si ocurre
```

Por ejemplo:

```
import os

try:
    os.mkdir("/root/nueva_carpeta")
except PermissionError:
    print("No tiene permisos para crear el directorio")
```

2.6.5 5. Tratar varias excepciones

Podemos utilizar varios bloques `except` .

```
import os

ruta = input("Directorio a crear: ")

try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos para crear el directorio")

except FileNotFoundError:
    print("Algún directorio intermedio no existe")
```

Cada `except` trata un tipo de excepción diferente.

2.6.6 6. Obtener información sobre la excepción

Podemos guardar el objeto de la excepción en una variable. Vea un ejemplo:

```
try:
    os.mkdir("/root/nueva_carpeta")

except PermissionError as e:
    print("Error de permisos")
    print("Detalle:", e)
```

La variable `e` contiene información generada por Python sobre la excepción.

2.6.7 7. Captura genérica con `Exception`

También podemos capturar la excepción sin especificar el tipo usando `Exception` . Sería similar a un "caso por defecto";

```
try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos")

except Exception as e:
    print("Se ha producido otro error:", e)
```

> ⚠ **Las excepciones específicas deben ir primero**
>
> Los casos específicos deben aparecer antes que `Exception` , dejando el caso genérico al final.

Usar sólo una excepción genérica puede ocultar información útil. Es preferible capturar excepciones concretas cuando sabemos cuáles pueden ocurrir y utilizar `Exception` solo como último caso cuando tenga sentido.

2.6.8 8. Ejemplos excepciones

8.1. `input()` y `ValueError`

```
try:
    edad = int(input("Edad: "))
    print("Edad:", edad)

except ValueError:
    print("Debe introducir un número entero")
```

Si el usuario escribe un texto que no se puede convertir con `int()`, se produce `ValueError`.

8.2. división por cero

```
try:
    a = int(input("Dividendo: "))
    b = int(input("Divisor: "))

    resultado = a / b
    print("Resultado:", resultado)

except ValueError:
    print("Debe introducir números enteros")

except ZeroDivisionError:
    print("No se puede dividir entre cero")
```

Aquí pueden ocurrir dos problemas distintos:

- el usuario introduce algo que no es un entero;
- el divisor es `0`.

2.6.9 9. Anexo I: `else` y `finally`

9.1. else

Se puede añadir un bloque `else` a `try` qeu se ejecuta **solo si no se produce ninguna excepción**.

```
try:
    numero = int(input("Número: "))

except ValueError:
    print("Valor incorrecto")

else:
    print("Número correcto:", numero)
```

9.2. `finally`

El bloque `finally` se ejecuta **siempre**, tanto si se produce una excepción como si no.

```
try:
    numero = int(input("Número: "))
    print("Número:", numero)

except ValueError:
    print("Valor incorrecto")

finally:
    print("Fin del intento")
```

`finally` es útil para tareas de limpieza que deben ejecutarse siempre.

9.3. Estructura completa

```
try:
    ...
```

```
except TipoError:
    ...
else:
    ...
finally:
    ...
```

El significado es:

- `try` : código que puede generar una excepción;
- `except` : qué hacer si ocurre;
- `else` : qué hacer si todo ha ido bien;
- `finally` : qué hacer siempre.

No es necesario utilizar todos los bloques.

2.6.10 10. Ejemplo completo con directorios

```
import os

ruta = input("Directorio a crear: ")

try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos para crear el directorio")

except FileNotFoundError:
    print("Algún directorio intermedio no existe")

except Exception as e:
    print("Se ha producido un error no previsto:")
    print(e)

else:
    print("Directorio creado correctamente")

finally:
    print("Fin del intento de creación")
```

2.6.11 11. Anexo II: Excepciones habituales en Python

Excepción	Ejemplo
`ValueError`	`int("hola")`
`ZeroDivisionError`	`10 / 0`
`IndexError`	acceder fuera de una lista
`KeyError`	usar una clave inexistente
`FileNotFoundError`	abrir un fichero inexistente
`FileExistsError`	crear un directorio que ya existe
`PermissionError`	acceder sin permisos

11.1. `IndexError` con listas

```
puertos = [22, 80, 443]

try:
    posicion = int(input("Posición: "))
    print(puertos[posicion])

except ValueError:
    print("Debe introducir un número")
```

```
except IndexError:
    print("La posición no existe")
```

11.2. `KeyError` con diccionarios

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20"
}

try:
    print(servidor["puerto"])

except KeyError:
    print("La clave no existe")
```

Para una consulta sencilla también podemos utilizar:

```
servidor.get("puerto")
```

o comprobar previamente:

```
if "puerto" in servidor:
```

> **No todo debe resolverse con `try`**
>
> A veces existe una forma más sencilla de evitar el problema.

2.6.12 12. Anexo III: Recomendaciones

12.1. Evitar bloques `try` demasiado grandes

Es preferible limitar el `try` al código que realmente puede generar la excepción que queremos controlar.

Por ejemplo:

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Edad incorrecta")
```

resulta más claro que incluir muchas operaciones no relacionadas dentro del mismo `try`.

12.2. Evitar `except:` sin indicar excepción

Python permite:

```
except:
    print("Error")
```

pero normalmente conviene evitar esta forma. Es preferible:

```
except ValueError:
```

o si necesitamos un caso genérico:

```
except Exception as e:
```

2.7 Ejercicios II

2.7.1 Ejercicios

Ejercicio 1. Directorio actual

Utiliza el módulo `os` para mostrar el directorio actual.

Ejercicio 2. Contenido de `/tmp`

Obtén el contenido de `/tmp` con `os.listdir()` y listas el contenido con un bucle `for`.

Ejercicio 3. Crear directorio

Pide al usuario un nombre y crea dentro de `/tmp` un directorio con ese nombre.

Ejercicio 4. Renombrar un archivo

Suponga que existe `/tmp/informe.txt`. Utiliza `os.rename()` para renombrarlo como `/tmp/informe.old`.

2.7.2 Ejercicios

Ejercicio 1. Información de un servidor

Crear una función:

```
muestra_servidor(servidor)
```

que recibe un diccionario con claves `nombre` e `ip` de un equipo y los muestra por pantalla, cada uno en una línea.

Ejercicio 2. Parámetro opcional

Modificar el ejercicio anterior para añadir al diccioanrio la clave `activo` de tipo lógico. La función se modifica para mostrar en su salida la cadena "Up" o "Down" en función de dicha clave.

Ejercicio 3. Validar un puerto

Crear la función:

```
puerto_valido(puerto)
```

que devuelva `True` si el puerto está entre `1` y `65535` y `False` en caso contrario.

Más ejercicios

1. Función que devuelve `True` si un número entero está entre 1 y 10, `False` en caso contrario.
2. Función que solicita un número entero al usuario. Si el valor no está entre 1 y 10 solicita de nuevo el número. Devuelve como resultado de la función el número.
3. Función que recibe como argumento `login` y `password`. Devuelve `True` si la clave es correcta o `False` si no (si la longitud es menor que 10, o el `login` incluido en el `password`)

Y otra tanda

1. Solicitar una IP por teclado. Comprobar que no esté incluida en la lista `bloqueados = ["10.10.10.100", "192.168.1.200", "8,8,8,8"]`.
2. Solicitar por teclado un string con números de puertos separados por espacios. Crear a partir de esa entrada una lista de puertos.
3. Dado el diccionario `servicios = {"ssh": 22, "http":80, "ftp": 21}` escriba una función que use un bucle for para imprimir solo los nombres de los servicios (las claves).
4. Usando el mismo diccionario crear función que use el metodo `.items()` para imprimir para cada servicio `El servicio xxx usa el puerto yyy`.

2.7.3 Ejercicios

1. Ejercicio 1. Conversión a entero

Pide al usuario un número entero.

Si introduce un valor que no pueda convertirse con `int()`, muestra:

```
Debe introducir un número entero
```

2. Ejercicio 2. División

Pide dos números enteros y divide el primero entre el segundo.

Controla:

- `ValueError`;
- `ZeroDivisionError`.

3. Ejercicio 3. Crear un directorio

Pide al usuario el nombre de un directorio y créalo con:

```
os.mkdir()
```

Controla al menos:

- `FileExistsError`;
- `PermissionError`.

Si se crea correctamente, muestra:

```
Directorio creado
```

4. Ejercicio 4. Acceso a una lista

Partiendo de:

```
servicios = ["ssh", "http", "https"]
```

pide una posición y muestra el servicio correspondiente.

Controla:

- `ValueError`;
- `IndexError`.

5. Ejercicio 5. Diccionario de servidor

Partiendo de:

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20",
    "puerto": 80
}
```

pide una clave al usuario e intenta mostrar su valor utilizando:

```
servidor[clave]
```

Controla `KeyError`.

2.8 Prácticas

2.8.1 Práctica Bloque 01

Fichero con datos: servidores.py

1. Apartados

- Apartado 01
- Apartado 02
- Apartado 03
- Apartado 04
- Apartado 05
- Apartado 06
- Apartado 07
- Apartado 08

2.8.2 Práctica: Control de catálogo

1. Situación de partida

Realizar una herramienta de consola para **consultar y supervisar un catálogo de productos**. La información no estará escrita dentro de tu programa: se obtendrá de una API REST pública llamada **DummyJSON**. A partir de los datos recuperados desarrollarás, paso a paso, un programa para listar productos, buscarlos, filtrarlos, calcular estadísticas, etc. La práctica repasa estructuras de control, listas, diccionarios, funciones y excepciones.

> **novedades**
>
> Se introducen únicamente dos novedades: la petición HTTP con `requests` y la conversión de JSON a estructuras Python mediante `.json()`.

1.1. OBJETIVOS DE APRENDIZAJE

Al terminar deberías poder:

- Recorrer y transformar **listas de diccionarios**.
- Acceder a **diccionarios y listas anidados**.
- Diseñar funciones que reciben y devuelven estructuras de datos.
- Validar datos y controlar **excepciones**.

2. Preparación

2.1. REQUISITO BIBLIOTECA `requests`

Es posible que el módulo `requests` está ya instalado. Lo puede comprobar con esta secuencia:

```
$ python3
Python 3.13.5 (main, Aug 10 2026, 12:06:59) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import requests
>>> quit()
$
```

Si no desde terminal, preferiblemente en un entorno virtual:

```
python3 -m venv venv
source venv/bin/activate
pip install requests
```

2.2. USO API DUMMYJSON

Comprueba primero en el navegador o con `curl` que esta dirección devuelve un documento JSON:

https://dummyjson.com/products?limit=3

El **diccionario** tiene una estructura parecida a esta (ejemplo abreviado):

```
{
  "products": [
    {
      "id": 1,
      "title": "Producto de ejemplo",
      "category": "beauty",
      "price": 9.99,
      "stock": 5,
      "rating": 4.4,
      "tags": ["beauty", "mascara"],
      "dimensions": {"width": 23.17, "height": 14.43, "depth": 28.01},
      "reviews": [
        {"rating": 5, "reviewerName": "Cliente A", "comment": "Muy bien"},
        {"rating": 2, "reviewerName": "Cliente B", "comment": "Mejorable"}
      ]
    }
  ],
  "total": 194,
  "skip": 0,
```

```
  "limit": 30
}
```

Observe que:

- El diccionario tiene 4 campos, el primero es `products`
- `products` es una **lista** de productos.
- Cada producto es un **diccionario**.
- Un producto contine entre otros
  - una lista: `tags`
  - un diccionario: `dimensions`
  - una lista de diccionarios: `reviews`
  - etc

Si lo desea puede consultar la documentación oficial de productos.

3. Apartados

3.1. APARTADO 1. CONEXIÓN CON LA API

Se le proporciona esta función ya terminada, y otra que simula la consulta por si no funciona la API o no tiene acceso a internet.

def obtener_productos

```python
def obtener_productos(limite=10):
    """Consulta DummyJSON y devuelve la lista 'products'.

    Propaga RequestException si falla HTTP o la conexión y ValueError si
    la respuesta no contiene la estructura esperada.
    """
    if not isinstance(limite, int) or isinstance(limite, bool) or limite <= 0:
        raise ValueError("El límite debe ser un entero positivo")

    respuesta = requests.get(
        URL_PRODUCTOS,
        params={"limit": limite},
        timeout=10,
    )
    respuesta.raise_for_status() #  Si servidor devuelve error lanza excepción HTTPError
    datos = respuesta.json()

    if not isinstance(datos, dict) or not isinstance(datos.get("products"), list):
        raise ValueError("Respuesta inesperada: falta la lista 'products'")

    return datos["products"]
```

En ela puede observar que

1. Realiza una petición `GET` a `https://dummyjson.com/products` utilizando `requests.get()`; envía `limite` como parámetro `limit`.
2. Establece un tiempo de espera (`timeout`) de 10 segundos.
3. Comprueba si la respuesta HTTP indica un error mediante `raise_for_status()`.
4. Convierte la respuesta JSON en estructuras Python mediante `.json()`.
5. Comprueba que la respuesta sea un diccionario que contenga la clave `products` y que su valor sea una lista. Si no lo es, lanza `ValueError` con un mensaje claro.
6. Devuelve **solo la lista** de productos.

3.2. APARTADO 2. LISTAR PRODUCTOS

Implementa:

```python
def mostrar_productos(productos):
    ...
```

Muestra, para cada producto, su `id`, `title`, `category`, `price` y `stock` en una tabla de texto legible. Si recibes una lista vacía, muestra `No hay productos que mostrar.`

Ejemplo de formato (valores ficticios):

```
 ID  PRODUCTO                      CATEGORÍA            PRECIO     STOCK
  1  Teclado USB                   accessories           25.90         8
  2  Monitor 24 pulgadas           electronics          159.00         3
```

Puedes limitar visualmente la anchura del título sin cambiar su valor real en el diccionario.

??? tip "Pista: presentación" Utiliza un `for` y cadenas con formato (`f"{valor:<20}"` , `f"{precio:>8.2f}"`). La función muestra datos; no necesita devolver ninguna lista nueva.

3.3. APARTADO 3. BUSCAR Y FILTRAR

Implementa dos funciones que **devuelvan nuevas listas de referencias a productos**; no imprimirán nada y no modificarán el catálogo:

```python
def buscar_por_titulo(productos, texto):
    ...

def filtrar_productos(productos, categoria=None, precio_max=None, solo_disponibles=False):
    ...
```

Búsqueda: `buscar_por_titulo` devuelve productos cuyo título contiene el texto indicado, sin distinguir mayúsculas de minúsculas. Una cadena vacía puede devolver todos los productos.

Filtros combinables: `filtrar_productos` permite seleccionar por categoría (coincidencia exacta, sin distinguir mayúsculas/minúsculas), precio máximo (incluido en el rango) y disponibilidad (`stock > 0`).

Todos los filtros activos se combinan mediante una condición lógica **Y**.

Ejemplo de uso:

```python
resultado = buscar_por_titulo(productos, "phone")
mostrar_productos(resultado)

resultado = filtrar_productos(
    productos,
    categoria="beauty",
    precio_max=30,
    solo_disponibles=True
)
mostrar_productos(resultado)
```

No presupongas cuántos resultados habrá: los productos publicados en la API pueden cambiar.

??? tip "Pista: comparación de cadenas" Puedes utilizar `.lower()` (o `.casefold()`) y `.strip()` para normalizar texto. Construye una lista vacía, recorre los productos y añade con `.append()` los que cumplan todos los criterios aplicables.

3.4. APARTADO 4. ESTADÍSTICAS

Implementa:

```python
def calcular_estadisticas(productos):
    ...
```

Debe **devolver un diccionario** con estas claves:

Clave	Contenido
`total`	Cantidad de productos analizados
`precio_medio`	Media aritmética de precios, redondeada a 2 decimales
`mas_caro`	Diccionario del producto de precio más alto (o `None` si no hay productos)
`unidades_en_stock`	Suma de las existencias de todos los productos
`por_categoria`	Diccionario `categoría: número de productos`

La función debe funcionar con cualquier lista, **también vacía**. En ese caso devuelve cero para las cantidades y la media, `None` para `mas_caro` y `{}` para `por_categoria` .

Después crea una pequeña función de presentación o muestra el resultado desde el menú. No introduzcas llamadas a la API en esta función.

??? tip "Pista: contar por categorías" Piensa en un diccionario que se inicializa vacío. Cada vez que aparece una categoría, aumenta su contador. Evita dividir entre cero cuando no hay productos.

4. Datos para cálculos independientes Internet

Puedes copiar temporalmente esta lista dentro de tu script y llamar directamente a tus funciones para comparar resultados

```
PRODUCTOS_EJEMPLO = [
    {
        "id": 1, "title": "Router", "category": "redes",
        "price": 80.0, "stock": 2, "rating": 2.5,
        "tags": ["wifi", "redes"],
        "dimensions": {"width": 20, "height": 5, "depth": 15},
        "reviews": [{"rating": 5}, {"rating": 3}, {"rating": 4}],
    },
    {
        "id": 2, "title": "Switch", "category": "redes",
        "price": 120.0, "stock": 0, "rating": 4.1,
        "tags": [], "dimensions": {}, "reviews": [],
    },
    {
        "id": 3, "title": "UPS", "category": "energia",
        "price": 100.0, "stock": 9, "rating": 4.5,
        "tags": ["rack"], "dimensions": {},
        "reviews": [{"rating": 2}, {"rating": 5}],
    },
]
```

5. Recursos

- DummyJSON — Documentación de productos
- Documentación de `requests`
- Tutorial oficial de estructuras de datos en Python

3. Tools

3.1 Introducción a Python - Linters

3.1.1 1. Introducción

Hay determinados problemas en el código que pueden detectarse **antes de ejecutar el programa**:

- errores de sintaxis;
- nombres que no existen;
- módulos importados pero no utilizados;
- variables creadas pero no usadas;
- código poco legible o difícil de mantener;
- uso de malas prácticas;
- estilos de código inconsistentes.

Las herramientas que analizan el código buscando este tipo de problemas se denominan *linters*. En Python existen distintas herramientas, entre ellas:

- Flake8
- Pylint
- Pyflakes
- Ruff
- ...

En este tema utilizaremos principalmente **Flake8**. Flake8 reúne comprobaciones procedentes de herramientas como:

- **Pyflakes**, para detectar determinados errores en el código;
- **pycodestyle**, para comprobar convenciones de estilo;
- **McCabe**, para medir complejidad ciclomática.

Entre otras cosas puede detectar imports no utilizados, nombres no definidos, problemas de espaciado, líneas demasiado largas y funciones excesivamente complejas.

> **¿Qué hace un linter?**
>
> Un linter realiza **análisis estático** del código: examina el código fuente sin necesidad de ejecutarlo.
>
> Un linter puede detectar muchos problemas, pero no puede garantizar que el programa funcione correctamente. Para comprobar el comportamiento del programa necesitamos técnicas como el **testing**.

1.1. Linter, formatter y testing

Conviene distinguir tres tipos de herramientas:

Herramienta	Objetivo
Linter	Detectar problemas, malas prácticas y errores de estilo
Formatter	Modificar automáticamente el formato del código
Testing	Comprobar que el programa produce los resultados esperados

Por ejemplo:

- `flake8` es principalmente un **linter**;
- `black` es un **formatter**;
- `pytest` permite realizar **tests**.

Estas herramientas se complementan entre sí.

A continuación:

1. se aprte de un fichero con errores: `test_linters_inicial.py`;
2. se crea un entorno virtual;
3. y se instala `flake8`;
4. se prueba el linter desde la línea de comandos;
5. y se utiliza integrado con el editor Visual Studio Code;

3.1.2 2. Instalación

2.1. Entorno virtual

- Crear y activar

```
calidad$ python -m venv venv
calidad$ source venv/bin/activate
(venv) calidad$
```

- Instalar `flake8`

```
(venv) @tos:~/calidad$ pip install flake8
```

3.1.3 3. Ejecución en CLI

Se el indica a `flake8` el fichero que desea revisar:

```
(venv) calidad$ flake8 test_linters_inicial.py
test_linters_inicial.py:8:1: F401 'io' imported but unused
test_linters_inicial.py:10:1: F403 'from math import *' used; unable to detect undefined names
test_linters_inicial.py:14:1: E303 too many blank lines (3)
test_linters_inicial.py:28:5: F841 local variable 'some_global_var' is assigned to but never used
test_linters_inicial.py:30:15: E225 missing whitespace around operator
test_linters_inicial.py:41:80: E501 line too long (80 > 79 characters)
test_linters_inicial.py:51:10: E711 comparison to None should be 'if cond is not None:'
test_linters_inicial.py:55:25: E703 statement ends with a semicolon
test_linters_inicial.py:63:23: E275 missing whitespace after keyword
test_linters_inicial.py:63:24: E201 whitespace after '('
test_linters_inicial.py:66:13: E303 too many blank lines (2)
test_linters_inicial.py:72:58: E251 unexpected spaces around keyword / parameter equals
test_linters_inicial.py:74:28: E221 multiple spaces before operator
...
(venv) calidad$
```

Observe como cada mensaje tiene un foramto similar `fichero:línea:columna: código descripción`

- el **fichero**,
- **línea** y columna
- el **código del error**
- y una **breve descripción** de ese código de error

Por ejemplo, en la línea 28 detecta el error F481: a la variable `some_global_var` se le asigna un valor pero esa variable no es usada posteriormente.

El programa puede seguir funcionando

Muchos avisos de un linter no provocan necesariamente un error durante la ejecución.

Una variable no utilizada, por ejemplo, no impide que el programa funcione, pero puede indicar código innecesario o un olvido del programador.

Hay también avisos de *estilo*: líneas de más de 79 caracteres, demasiados espacios después de un operador, etc.

Los códigos de error se pueden consultar en la documentación oficial. Y en este enlace tiene ejemplos descriptivos de los mismos.

3.1.4 4. Algunos códigos habituales

No corregir sin pensar

Un linter **señala problemas**, pero normalmente es el programador quien decide cómo corregirlos.El objetivo no es conseguir que desaparezcan todos los mensajes a cualquier precio.

Hay que comprender qué está señalando cada aviso y decidir cuál es la corrección adecuada.

- F401 - Import no utilizado

```
import os
```

Si posteriormente no se utiliza `os` muestra `F401 'os' imported but unused`

- F841 - Variable local asignada pero no utilizada

```
def ejemplo():
    resultado = 10
    print("Fin")
```

Si `resultado` nunca se utiliza muestra `F841 local variable 'resultado' is assigned to but never used` ```

- E225 - Falta espacio alrededor de un operador

No recomendado:

```
total=precio*cantidad
```

Recomendado:

```
total = precio * cantidad
```

- E501 - Línea demasiado larga

Podemos dividir expresiones largas utilizando paréntesis:

```
mensaje = (
    "Este texto se ha dividido "
    "en varias líneas del código."
)
```

Analizar un directorio completo

Puede analizar el directorio actual:

```
flake8 .
```

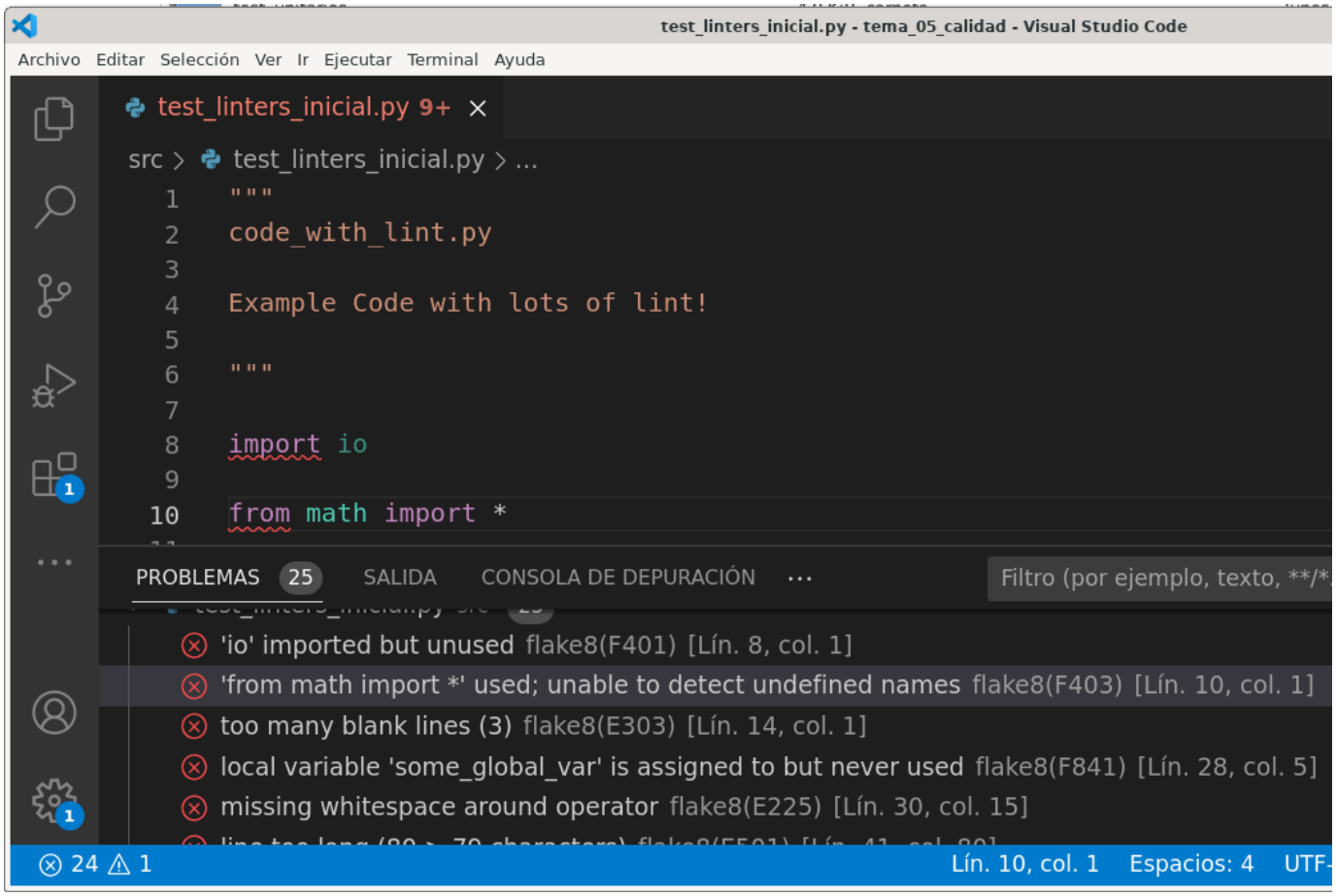
O una carpeta concreta:

```
flake8 src/
```

3.1.5 5. Usando el linter en el editor

Visual Studio Code dispone actualmente de una extensión **Flake8** publicada por Microsoft. Una vez instalada:

- los problemas aparecen directamente en el editor;
- también aparecen en el panel **Problemas**;
- el análisis puede ejecutarse automáticamente al trabajar con archivos Python.



Configuración antigua de VS Code

En versiones antiguas se elegía el linter mediante `Python: Select Linter` que debái estar previamente instalado.

Ese procedimiento está obsoleto. La extensión actual incluye su propia versión de Flake8, aunque también puede configurarse para usar una instalación del entorno del proyecto.

Para este curso es suficiente recordar:

1. instalar Flake8 en el entorno virtual para usarlo desde terminal;
2. instalar la extensión **Flake8** de Microsoft para usarlo desde VS Code.

3.1.6 6. Seleccionar reglas

Podemos revisar solo determinados códigos: use `--select` y se le indican los códigos a comprobar:

```
flake8 --select E225,E501 test_linters_inicial.py
```

También puede seleccionar grupos:

```
flake8 --select E,W .
```

3.1.7 7. Ignorar reglas

De forma similar, puede **ignorar** códigos con la opción `--ignore` :

```
flake8 --ignore E501,E303 test_linters_inicial.py
```

3.1.8 8. Ignorar una línea

Es posible ignorar un aviso en una línea concreta:

```
import modulo_no_usado  # noqa
```

> ⚠ **`noqa` no sustituye a corregir el código**
>
> Debe utilizarse solo cuando exista una razón clara para mantener esa línea.

3.1.9 9. Configuración en fichero

Tiene por otra parte la posibilidad de crear un fichero de configuración. Flake8 admite configuración en:

- `.flake8`
- `setup.cfg`
- `tox.ini`

La configuración se coloca en una sección:

```
[flake8]
```

Un posible fichero `tox.ini` en la raiz del proyecto podría ser:

tox.ini

```
[flake8]
ignore = E501,E303
exclude = .git,__pycache__,old,build,dist
max-line-length = 100
max-complexity = 10
```

Después basta ejecutar:

```
flake8 .
```

De forma simplificada, la prioridad de la configuración es:

1. las opciones de línea de comandos tienen prioridad;
2. después se usa la configuración del proyecto;
3. finalmente se aplican los valores por defecto.

3.1.10 10. Ejercicio

Utilizar la herramienta `flake8` con el código de los ejemplos y ejercicios de clase. Revisar y corregir los errores que muestra.

3.1.11 11. Enlaces

- Documentación oficial flake8
- Opciones de flake8: https://flake8.pycqa.org/en/latest/user/options.html

- Códigos de error: https://flake8.pycqa.org/en/latest/user/error-codes.html
- **Descripción con ejemplos** de los códigos de error: https://www.flake8rules.com/

3.1.12 12. Anexo: linter pylint

El objetivo es conocer los linters y su uso básico, y con uno de ellos es suficiente. Pero si lo desea puede probar `pylint` también.

Se instala y se invoca de forma similar a `flake8`

```
(venv) calidad$ pip install pylint
(venv) calidad$ pylint test_linters_inicial.py
************* Module test_linters_inicial
test_linters_inicial.py:55:0: W0301: Unnecessary semicolon (unnecessary-semicolon)
test_linters_inicial.py:63:0: C0325: Unnecessary parens after 'return' keyword (superfluous-parens)
test_linters_inicial.py:66:0: C0325: Unnecessary parens after 'return' keyword (superfluous-parens)
....
-----------------------------------
Your code has been rated at 0.00/10
(venv) calidad$
```

Si desea más información sobre el uso de `pylint` :

- https://pylint.pycqa.org/en/latest/user_guide/run.html
- https://learn.adafruit.com/improve-your-code-with-pylint/pylintrc

3.2 Introducción a Python - Introducción al testing

El **testing** consiste en comprobar de forma automática que nuestro programa se comporta como esperamos.

En lugar de ejecutar el programa manualmente muchas veces y comprobar cada resultado a mano, podemos escribir pruebas que:

1. ejecuten una función con unos datos determinados;
2. comparen el resultado obtenido con el esperado;
3. indiquen si la prueba ha sido correcta o ha fallado.

En este tema utilizaremos principalmente **pytest**, aunque al final se incluye una breve introducción a `unittest`, que forma parte de la biblioteca estándar de Python.

3.2.1 1. Módulo pytest

`pytest` es un framework de pruebas para Python que permite escribir y ejecutar tests de forma sencilla y con poco código repetitivo.

1.1. Instalación de pytest

Es recomendable trabajar dentro de un entorno virtual para instalar `pytest` sin modificar los paquetes globales del sistema.

```
clases@ssd:~/testing$ python3 -m venv venv
clases@ssd:~/testing$ source venv/bin/activate
(venv) clases@ssd:~/testing$
```

Instalamos y verificamos:

```
(venv) clases@ssd:~/testing$ pip install pytest
(venv) clases@ssd:~/testing$ pytest --version
pytest 9.1.1
```

Entorno virtual

Cuando está activado suele aparecer `(venv)` al comienzo del prompt.

Para salir del entorno virtual:

```
deactivate
```

1.2. Código que vamos a probar

Supongamos que tenemos este fichero:

```python
# File: seguridad.py

def comprueba_clave(login, password):
    valida = True

    claves_prohibidas = {
        "123456789012",
        "password12345",
        "administrador2026"
    }

    if len(password) < 10:
        valida = False
    elif password in claves_prohibidas:
        valida = False
    elif login in password:
        valida = False

    return valida
```

La función devuelve `True` si la contraseña se considera válida y `False` en caso contrario.

Las reglas implementadas son:

- la contraseña debe tener al menos 10 caracteres;
- no puede estar en el conjunto de claves prohibidas;
- no puede contener el login del usuario.

1.3. Fichero de test

Para que `pytest` descubra automáticamente las pruebas:

- los archivos suelen comenzar por `test_`;
- las funciones de prueba deben comenzar por `test_`;
- utilizamos `assert` para comparar el resultado obtenido con el esperado.

```
# File: test_seguridad.py

import pytest
from seguridad import comprueba_clave


def test_clave_valida():
    """Clave válida."""
    assert comprueba_clave(
        "usuario123",
        "ClaveMuySegura2026!"
    ) is True


def test_clave_corta_menos_de_10_caracteres():
    """Clave con menos de 10 caracteres."""
    assert comprueba_clave(
        "usuario",
        "12345679"
    ) is False
```

1.4. La instrucción `assert`

La forma general es:

```
assert resultado_obtenido == resultado_esperado
```

Si la expresión es verdadera, la prueba pasa. Si es falsa, falla.

Por ejemplo:

```
assert comprueba_clave("usuario", "12345679") is False
```

significa:

> Espero que esta llamada devuelva `False`.

Para valores booleanos resulta habitual escribir:

```
is True
```

o:

```
is False
```

1.5. Ejecución

Podemos ejecutar todos los tests descubiertos por `pytest`:

```
pytest
```

O un fichero concreto:

```
pytest test_seguridad.py
```

Con `-v` obtenemos más detalle:

```
pytest -v
```

Una ejecución correcta podría mostrar:

```
collected 2 items

test_seguridad.py ..                                [100%]

2 passed
```

Cada punto representa una prueba superada.

3.2.2 2. Diseñar buenos casos de prueba

No basta con comprobar un único caso. Conviene pensar en varios tipos de pruebas.

2.1. Casos válidos

Entradas que deberían aceptarse:

```
assert comprueba_clave(
    "juan",
    "ClaveSegura2026"
) is True
```

2.2. Casos inválidos

Entradas que deberían rechazarse:

```
assert comprueba_clave(
    "admin",
    "123456789"
) is False
```

2.3. Casos límite

Son especialmente importantes. Si la condición es:

```
if len(password) < 10:
```

conviene probar:

- 9 caracteres;
- exactamente 10;
- 11 caracteres.

> **Prueba los límites**
>
> Cuando una condición utiliza `<`, `<=`, `>` o `>=`, prueba valores justo por debajo, en el límite y justo por encima.

3.2.3 3. Conjuntos de datos de prueba

Podemos guardar varios casos en una lista de tuplas:

```
dataset_pruebas = [
    ("juan_perez", "ContrasenaValida123", True),
    ("admin_sys", "SistemasOperativos2026", True),
    ("admin", "", False),
```

```
    ("admin", "123456789", False),
]
```

Y recorrerlos:

```
def test_comprueba_clave_con_bucle():
    for login, password, esperado in dataset_pruebas:
        resultado_real = comprueba_clave(login, password)

        assert resultado_real == esperado, (
            f"Fallo -> login='{login}', "
            f"password='{password}'"
        )
```

3.1. Mensajes en `assert`

Podemos añadir un mensaje en assert que solo se mostrará si la prueba falla.

```
assert resultado == esperado, "Mensaje si falla"
```

3.2. Limitación del bucle

Para `pytest`, toda la función sigue siendo un único test. Si un `assert` falla, el bucle no continúa con el resto de casos. Por eso resulta muy útil la parametrización.

3.2.4 4. Pruebas parametrizadas

`pytest` permite ejecutar la misma función varias veces con diferentes datos:

```
@pytest.mark.parametrize(
    "login, password, resultado_esperado",
    [
        ("juan_perez", "ContrasenaValida123", True),
        ("admin_sys", "SistemasOperativos2026", True),
        ("admin", "1234", False),
        ("admin", "password", False),
    ],
)
def test_comprueba_clave_parametrizado(
    login,
    password,
    resultado_esperado
):
    assert(comprueba_clave(login, password)== resultado_esperado)
```

Cada tupla se convierte en un caso de prueba independiente. Ventajas de pruebas paramrtrizadas respecto al bucle:

- si uno falla, los demás siguen ejecutándose;
- `pytest` indica qué combinación ha fallado;
- evitamos repetir muchas funciones similares.

3.2.5 5. Detectar un error real

Probemos:

```
assert comprueba_clave(
    "administrador2026",
    "Administrador2026"
) is False
```

Esperamos que sea inválida porque la contraseña contiene el login.

Sin embargo, la función original utiliza:

```
elif login in password:
```

y esta comparación distingue mayúsculas y minúsculas. Por tanto el login `administrador2026` no coincide con `Administrador2026`

El test nos ha permitido descubrir un caso que la implementación original no contemplaba.

3.2.6 6. Corregir la comparación

Podemos normalizar ambos textos antes de compararlos:

```
def comprueba_clave(login, password):
    valida = True

    claves_prohibidas = {
        "123456789012",
        "password12345",
        "administrador2026"
    }

    login_normalizado = login.lower()
    password_normalizada = password.lower()

    if len(password) < 10:
        valida = False
    elif password_normalizada in claves_prohibidas:
        valida = False
    elif login_normalizado in password_normalizada:
        valida = False

    return valida
```

Normalizar antes de comparar

Convertir ambos textos a minúsculas permite hacer una comparación que no dependa de mayúsculas y minúsculas.

3.2.7 7. Ejercicios

1. Amplía los tests para comprobar todos los requisitos:
    - longitud mínima;
    - claves prohibidas;
    - presencia del login en la contraseña.
2. Añade el caso de login `administrador2026` y password `Administrador2026`. Comprueba que la primera versión de la función no se comporta como esperamos.
3. Modifica `comprueba_clave()` para ignorar diferencias entre mayúsculas y minúsculas.
4. Añade casos combinando mayúsculas y minúsculas.
5. Añade tests para contraseñas de:
    - 9 caracteres;
    - 10 caracteres;
    - 11 caracteres.
6. Convierte un test basado en un bucle en una prueba parametrizada.

3.2.8 8. Anexo: unittest

Python incluye de serie el módulo `unittest`, por lo que no es necesario instalarlo.

Las pruebas suelen escribirse como métodos de una clase que hereda de `unittest.TestCase`:

```
import unittest

from seguridad import comprueba_clave


class TestCompruebaClave(unittest.TestCase):

    def test_clave_valida(self):
        self.assertTrue(
            comprueba_clave(
                "usuario123",
                "ClaveMuySegura2026!"
            )
        )
```

```
    def test_clave_corta(self):
        self.assertFalse(
            comprueba_clave(
                "usuario",
                "12345679"
            )
        )


if __name__ == "__main__":
    unittest.main()
```

8.1. Métodos habituales

```
self.assertTrue(valor)
self.assertFalse(valor)
self.assertEqual(valor1, valor2)
self.assertNotEqual(valor1, valor2)
```

8.2. Conjuntos de datos con `subTest`

```
import unittest

from seguridad import comprueba_clave


class TestCompruebaClave(unittest.TestCase):

    def test_varios_casos(self):
        dataset = [
            (
                "juan_perez",
                "ContrasenaValida123",
                True
            ),
            ("admin", "1234", False),
            ("usuario", "password12345", False),
        ]

        for login, password, esperado in dataset:
            with self.subTest(
                login=login,
                password=password
            ):
                resultado = comprueba_clave(
                    login,
                    password
                )
                self.assertEqual(resultado, esperado)


if __name__ == "__main__":
    unittest.main()
```

`subTest()` permite distinguir mejor los diferentes casos de un mismo bucle de prueba: permite saber qué subcaso falló y di un subtest falla, los siguientes también se ejecutan. Es el equivalente práctico más cercano en `unittest` a las pruebas parametrizadas de `pytest`, aunque internamente no funcionan exactamente igual.

8.3. pytest o unittest

En este curso utilizaremos principalmente `pytest` porque permite escribir pruebas muy sencillas y facilita especialmente la parametrización. `unittest` también es importante porque:

- viene incluido con Python;
- se utiliza en muchos proyectos;
- no necesita paquetes adicionales.

Tenga en cuenta que con `unittest` + `subTest()` el resultado suele ser **menos detallado que con `pytest.mark.

> `subTest()` permite ejecutar varios casos dentro de un mismo método de `unittest` y saber cuál ha fallado, pero `pytest.mark.parametrize` convierte cada conjunto de datos en un caso de prueba independiente y ofrece una salida más detallada.

	`unittest` + `subTest()`	`pytest.mark.parametrize`
Dataset con 3 casos	1 test + 3 subtests	3 tests
Si uno falla	Continúa con los demás	Continúa con los demás
Identifica el caso que falla	Sí	Sí
Muestra los casos correctos individualmente	Normalmente no	Sí con `-v`
Salida fácil de interpretar	Menos	Más

En ese sentido **esta es una de las ventajas de `pytest`** : al trabajar con datasets, se ve más claramente qué casos pasan y cuáles fallan.

3.3 Python: Calidad y documentación del código

3.3.1 1. Documentación en Python

> *"Code is more often read than written."*
>
> **Guido van Rossum**

1.1. Comentar vs documentar el código

Hay que diferenciar documentar de comentar.

- Comentar el código
  - Propósito: Notas rápidas para el programador en el futuro o compañeros, desactivar código temporalmente, explicar decisiones de diseño.
  - Dónde: Directamente en el archivo fuente.
  - Cuándo: Solo cuando el código no es lo suficientemente claro por sí mismo, para lógica compleja, o por razones legales/licencias.
  - Evita comentarios obvios ("// sumar dos números") o desactualizados, que son ruido y engañan.
- Documentar el código
  - Propósito: Explicar el funcionamiento general, arquitectura, API, y cómo usar el software a otros desarrolladores o usuarios finales.
  - Dónde: Archivos README, Wikis,**herramientas de generación de documentación o comentarios en formato especial**.
  - Cuándo: Siempre es necesario para proyectos compartidos o públicos.
  - Herramientas: **Usar generadores que extraen información de comentarios bien formateados** para crear manuales.

En python los comentarios se inician con el carácter `#`

```
# TODO:
# pendiente añadir feature 123
```

Y la documentación se puede incorporar en el código usando los llamados "docstring" (ver anexo).

1.2. Generando la documentación

Gracias a los *docstrings* y sus formatos existe la posibilidad de generar la documentación de un proyecto, modulo, clase, etc. de forma automática a partir del código. Hay diferentes soluciones y una de ellas es el módulo `pdoc3` .

Para ver su funcionamiento debe instalar dicho modulo en un entorno virtual:

```
(venv) clases@d12:~/src_doc$ pip install pdoc3
Collecting pdoc3
  Downloading pdoc3-0.10.0-py3-none-any.whl (135 kB)
     |████████████████████████████████████████| 135 kB 3.6 MB/s
....
Installing collected packages: MarkupSafe, mako, zipp, importlib-metadata, markdown, pdoc3
Successfully installed MarkupSafe-2.0.1 importlib-metadata-4.8.2 mako-1.1.6 markdown-3.3.6 pdoc3-0.10.0 zipp-3.6.0
(venv) clases@d12:~/src_doc$pdoc3 --version
pdoc3 0.10.0
(venv) clases@d12:~/src_doc$
```

Si lo invoca con el módulo calculadora (o cualquier otro de los ejemplos previos) muestra en la salida estándar la documentación generada en base a los docstrings presentes en el código:

```
(venv) clases@d12:~//src_doc$ cat calculadora.py
def suma(a:int,b:int) ->int:
    return a + b

def resta(a,b):
    """ Resta dos numeros."""
    return a-b
(venv) clases@d12:~/src_doc$ pdoc3 calculadora
Module calculadora
==================

Functions
```

```
---------


`resta(a, b)`
:   Resta dos numeros.


`suma(a: int, b: int) -> int`
:
(venv) clases@d12:~/PPS/calidad/src_doc$
```

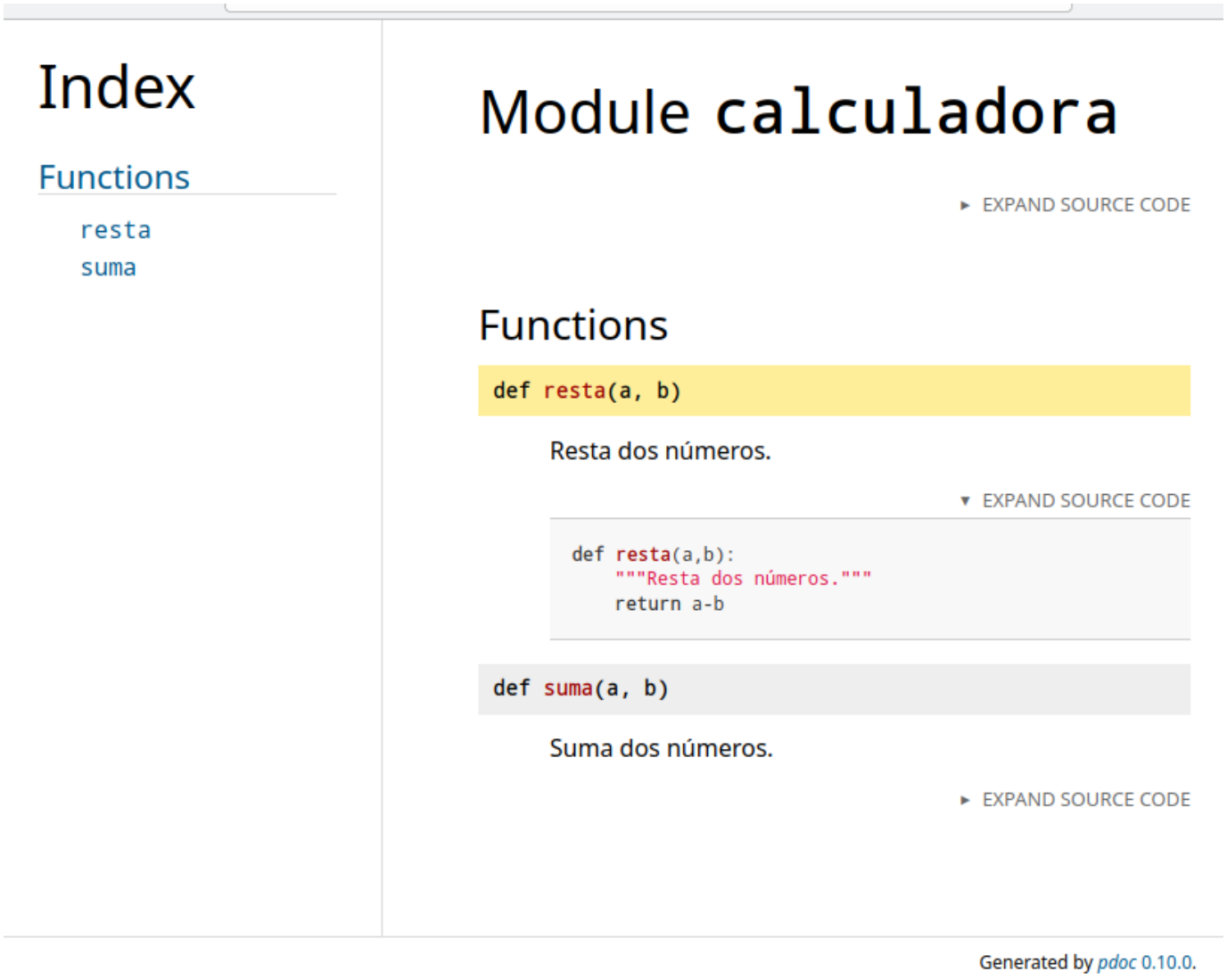
Es más interesante generar dicha documentación en formato HTML con la opción `--html`

```
(venv) @tos:~/docstring$ pdoc3 --html calculadora
html/calculadora.html
(venv) @tos:~/docstring$
```

Y abrirla en el navegador

```
(venv) @tos:~/docstring$ firefox html/calculadora.html
```

Observe que se generan títulos, un índice, documentación a partir de los docstring, etc. Ademaś permite ver el código con el desplegable *"Expand Source Code"*:



Observe un ejemplo de un módulo con una clase:

```
""" This module deals with all animals. So far, only dogs are implemented.

# TODO
 - Add the rest of the animals.
```

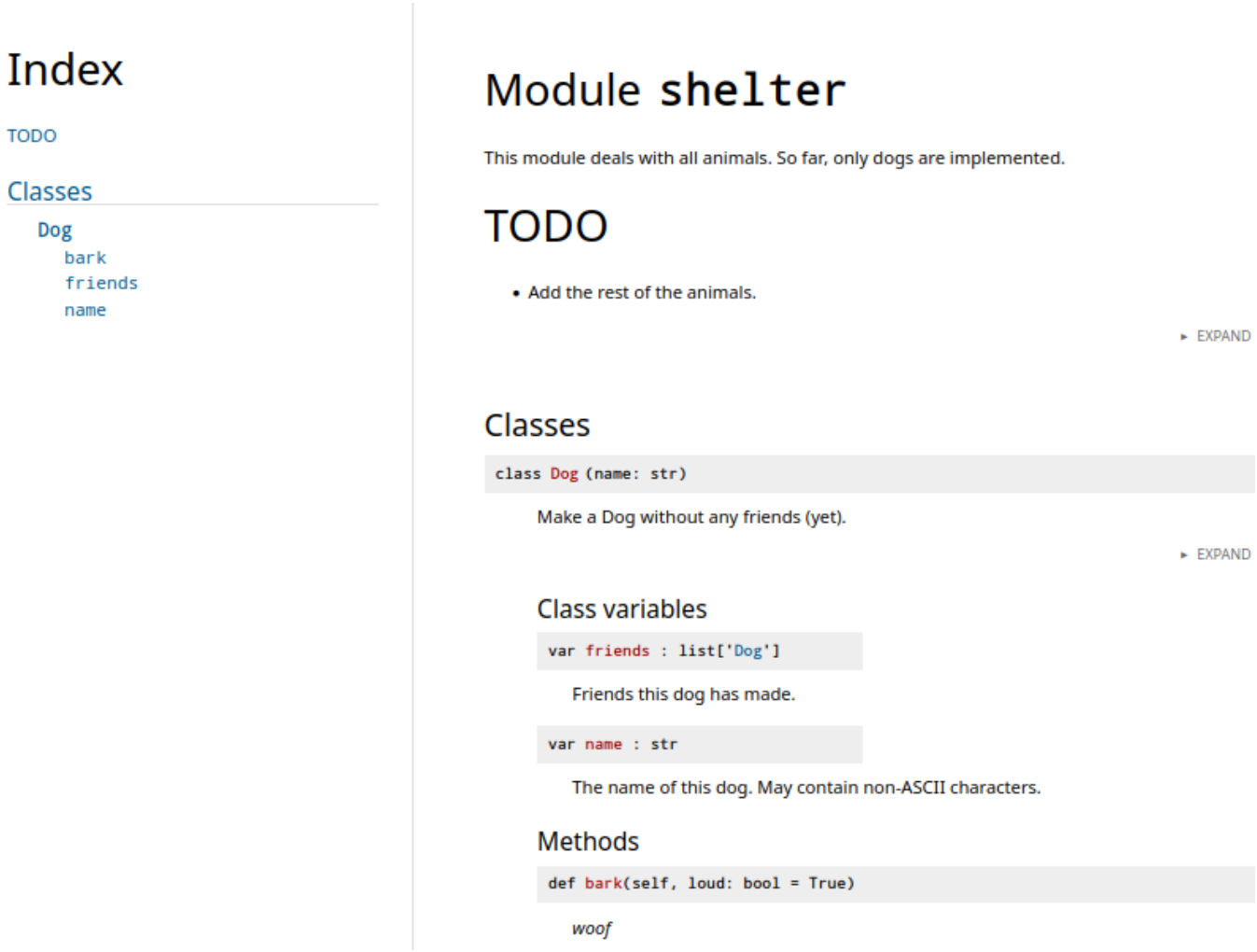
```
"""
from __future__ import annotations

class Dog:
    name: str
    """The name. May contain non-ASCII characters."""
    friends: list ["Dog"]
    """Friends this dog has made."""

    def __init__(self, name: str):
        """Make a Dog without any friends (yet)."""
        self.name = name
        self.friends = []

    def bark(self, loud: bool = True):
        """*woof*"""
```

Y cómo se genera la documentación por `pdoc3` en formato HTML:



Index

TODO

Classes

Dog
bark
friends
name

Module shelter

This module deals with all animals. So far, only dogs are implemented.

TODO

- Add the rest of the animals.

▸ EXPAND

Classes

class Dog (name: str)

Make a Dog without any friends (yet).

▸ EXPAND

Class variables

var friends : list['Dog']

Friends this dog has made.

var name : str

The name of this dog. May contain non-ASCII characters.

Methods

def bark(self, loud: bool = True)

woof

1.3. Comentar código con anotaciones *(Type Hinting)*

Es posible añadir información sobre los tipos de los argumentos o el valor de retorno en el propio código. Esto se logra con las anotaciones de tipo (**type hint**). Pero tenga en cuenta que estas anotaciones no implican comprobación de tipos al ejecutar.

Para añadir anotaciones de tipo - en los argumentos: tras el argumento el caracter `":"` y el tipo del argumento. - en el valor de retorno: tras el paréntesis de cierre de los argumentos una flecha `"->"` y el tipo de retorno.

Observe anotaciones en este ejemplo

```
# función sin tipos..
def hello(name):
    return(f"Hello {name}")

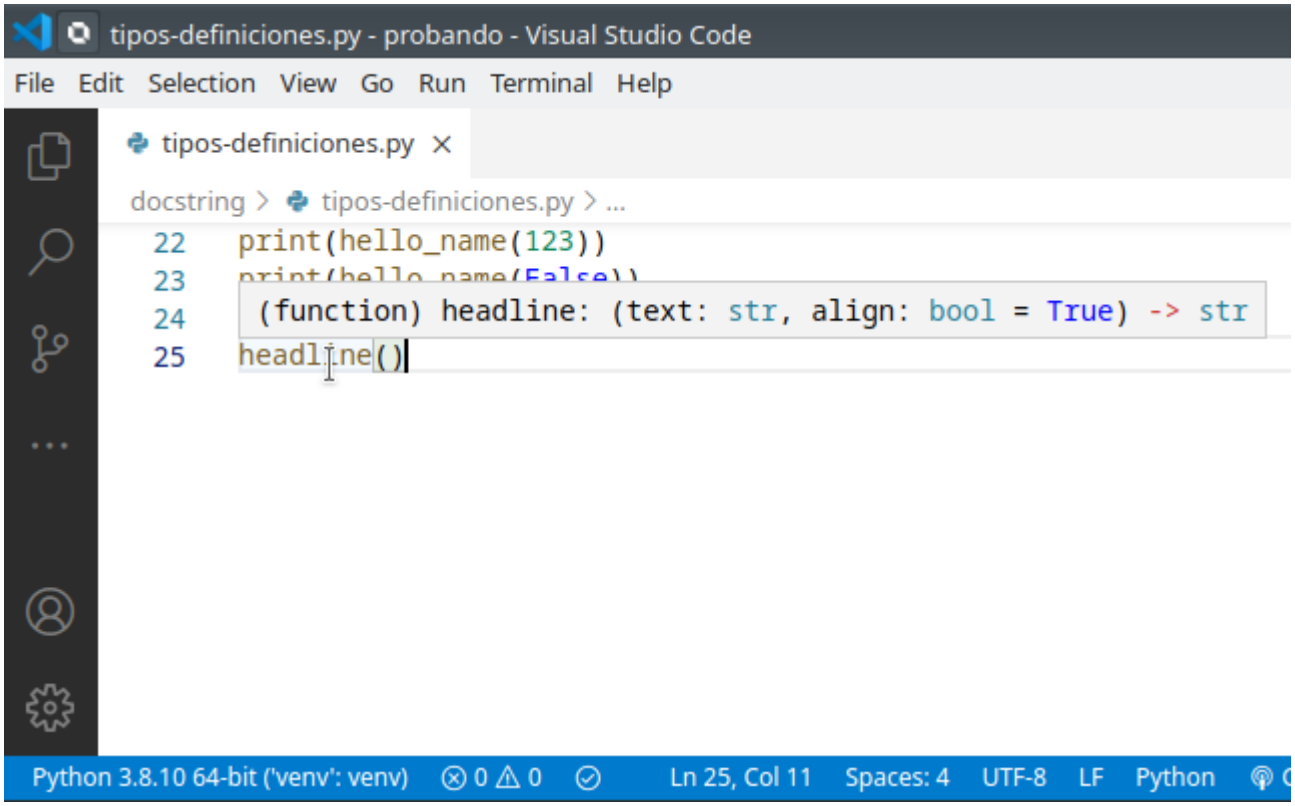
#funciones con tipos de parámetros y tipo retorno
```

```python
def hello_name(name: str) -> str:
    return(f"Hello {name}")

def headline(text: str, align: bool = True) -> str:
    if align:
        return f"{text.title()}\n{'-' * len(text)}"
    else:
        return f" {text.title()} ".center(50, "o")
```

Las anotaciones sirven para:

- Mejorar la documentación del código
- Que herramientas como un editor, a la hora de añadir una llamada a una función nos de una ayuda visual sobre la misma y los tipos esperados:



3.3.2 2. Calidad en el código

El código se considera de calidad si:

- Hace lo que se supone que debe hacer
- No contiene fallos o da problemas.
- **Es sencillo de leer, mantener y modificar para añadir mejoras.**

Es en ese tercer apartado en el que nos centramos en este documento.

2.1. Guías de estilo

La guía fundamental es la *Python Enhancement Proposal 8* o PEP-8 que presenta las directrices a seguir para escribir código Python.

Se indican cuestiones como: - Usar 4 espacios en el sangrado del códigoy no usar tabuladores. - Una longitud máxima de 79 caracteres en cada línea. - Sobre las líneas en blanco: - Rodear las funciones de primer nivel y clases con dos líneas en blanco - Los métodos dentro de una clase con una única linea en blanco - *Imports*: - Normalmente en líneas separadas - Primero los estándar, luego módulos de terceros, luego los propios - etc.

Adicionalmente hay otras guías de estilo especificas para empresas, por ejemplo la de Google.

2.2. Comentarios y documentación

Se ha trabajado este apartado previamente: uso de *docstrings*, *type hint* y la herramienta `pdoc3`.

3.3.3 3. Ejercicio: generar documentación

- Recuperar o crear el módulo `calculadora`.
- Documentar ese módulo con *docstrings*. Documentar los argumentos de las funciones y los valores de retorno de los dos métodos usando dos formatos distintos (ver anexo):
- formato de Google.
- formato de numpy.

Nota: Puede usar `pdoc3` en modo servidor con la opción `--http localhost:8080` y editar y ver los cambios en la documentación:

```
(venv) @tos:~/docstring$ pdoc3 --http localhost:8080 calculadora
Starting pdoc server on localhost:8080
pdoc server ready at http://localhost:8080
...
```

3.3.4 4. Enlaces adicionales

- PEP 257 – Docstring Conventions: https://www.python.org/dev/peps/pep-0257/
- Syntax and best practices for docstrings used with the numpy: https://numpydoc.readthedocs.io/en/latest/format.html
- Google Comments and Docstrings: https://google.github.io/styleguide/pyguide.html#38-comments-and-docstrings
- PEP 484 – Type Hints: https://peps.python.org/pep-0484/
- Documenting Python: A Complete Guide: https://realpython.com/documenting-python-code/?utm_source=pocket_mylist

3.3.5 5. Anexo *docstrings*

El interprete de comandos python proporciona ayuda sobre comandos, clases, etc. usando `help()`:

```
>>> help(str)
Help on class str in module builtins:

class str(object)
 |  str(object='') -> str
 |  str(bytes_or_buffer[, encoding[, errors]]) -> str
 |
 |  Create a new string object ....
 |  ....
>>>
```

De hecho puede ver todo la información relacionada con el objeto `str` con `dir()`

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
...
```

Para obtener ayuda del método `split`:

```
>>> help(str.split)

Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)
    Return a list of the substrings in the string, using sep as the separator string.
    ...
```

Dentro de esa información se encuentra el atributo `__doc__` que almacena esa información . Si accede a dicho atributo se muestra la ayuda de dicho objeto:

```
>>> print(str.__doc__)
str(object='') -> str
str(bytes_or_buffer[, encoding[, errors]]) -> str

Create a new string object from the given object. If encoding or
```

```
...
>>>
```

En el caso de `split`:

```
>>> print(str.split.__doc__)
Return a list of the substrings in the string, using sep as the separator string.

  sep
    The separator used to split the string.

  ...

>>>
```

Es decir, si tiene una clase o una función puede crear o modificar el atributo `.__doc__` y estaría generando el *docstring* de la ayuda.

En el siguiente ejemplo se crea una función y modifica `.__doc__`

```
>>> def hola(name):
...     print(f"hola {name}. Bienvenido")
...
>>> hola.__doc__ = "Funcion que saluda con nombre"
>>> help(hola)
Help on function hola in module __main__:

hola(name)
    funcion que saluda con nombre
(END)
>>>
```

Evidentemente este método de trabajo (modificar el atributo `__doc__`) no es práctico para documentar. En su lugar es preferible usar comentarios en el código **siguiendo un determinado formato**.

Observe en el siguiente ejemplo un comentario justo al inicio de la función. **Si lo añade de esa forma se le asigna de froma automática al atributo `.__doc__`:**

```
>>> def hola(name):
...     """ Funcion que saluda con parametro nombre."""
...     print(f"hola {name}. Bienvenido")
...
>>> help(hola)
Help on function hola in module __main__:

hola(name)
    Funcion que saluda con parametro nombre.
(END)
>>>
```

En el siguiente apartado se muestra que son los *docstrings* y cómo utilizarlos.

5.1. Docstrings línea/multilinea

> *A docstring is a string literal that occurs as the first statement in a module, function, class, or method definition. Such a docstring becomes the `__doc__` special attribute of that object.*
>
> Docstring conventions are described within PEP 257 (*Python Enhancement Proposals*) https://www.python.org/dev/peps/pep-0257/

Por lo tanto es un **literal que debe aparecer en la primera línea** del módulo, clase o función. Al aparecer en ese lugar se convierte en el atributo `__doc__` . Pueden ser de una línea o multilínea.

5.1.1. DOCSTRINGS DE UNA LÍNEA

Deben cumplir:

- Una única línea
- Empezar con mayúscula,
- Terminar en punto.
- Sin líneas en blanco ni antes ni después.

```
def hola(name):
    """ Funcion que saluda con parametro nombre."""
    print(f"hola {name}. Bienvenido")
```

5.1.2. DOCSTRINGS MULTILÍNEA

Si la ayuda necesita de más texto el formato debería ser:

- Una primera línea con el resumen de la ayuda
- Una línea en blanco
- Texto con la descripción en varias líneas
- Se cierra el literal
- Y una línea en blanco tras el literal

Observe este ejemplo tomado de una función especificando los parámetros y el valor de retorno:

```
"""Gets and prints the spreadsheet's header columns

Parameters
----------
file_loc : str
    The file location of the spreadsheet
print_cols : bool, optional
    A flag used to print the columns to the console (default is False)

Returns
-------
list
    a list of strings representing the header columns
"""
```

5.2. Clasificación por tipos

Se pueden clasificar los *docstrings* en función de que se apliquen a paquetes o módulos, clases o scripts. Por ejemplo, en el caso de los módulos se considera que deberían contener:

- un breve resumen de propósito y comportamiento
- Los métodos públicos con una breve descripción
- Las propiedades (atributos) de la clase
- ...

Tiene información más detallada en este enlace: https://realpython.com/documenting-python-code/#docstring-types

5.3. Clasificación por formatos o normas de estilo

Resulta que existen distintas "maneras" de organizar la información dentro del *docstring*. Algunos de estos formatos posibles son:

Formatting Type	Description
NumPy/SciPy docstrings	Combination of reStructured and GoogleDocstrings and supported by Sphinx
PyDoc	Standard documentation module for Python and supported by Sphinx
EpyDoc	Render Epytext as series of HTML documents and a tool for generating API documentation for Python modules based on their Docstrings
Google Docstrings	Google's Style

Vea como ejemplo la diferencia entre dos de ellos, NumPY y Google. Con el formato de Google el *docstring* de un método o función sería similar a:

```
"""Gets and prints the spreadsheet's header columns

Args:
    file_loc (str): The file location of the spreadsheet
    print_cols (bool): A flag used to print the columns to the console
```

```
        (default is False)

Returns:
    list: a list of strings representing the header columns
"""
```

Con Pynum la forma de especificar los parámetros y el valor de retorno es diferente:

```
"""Gets and prints the spreadsheet's header columns

Parameters
----------
file_loc : str
    The file location of the spreadsheet
print_cols : bool, optional
    A flag used to print the columns to the console (default is False)

Returns
-------
list
    a list of strings representing the header columns
"""
```

3.4 Depuración de scripts Python en Linux con VS Code

3.4.1 1. Introducción

La **depuración** o *debugging* es el proceso mediante el cual se analiza la ejecución de un programa para localizar y corregir errores. No todos los errores se presentan de la misma forma.

Al finalizar este tema el objetivo es ser capaz de:

- distinguir entre errores de sintaxis, errores en tiempo de ejecución y errores lógicos.
- utilizar `breakpoint()` y `pdb`.
- depurar scripts con Visual Studio Code.

1.1. Errores de sintaxis

Se producen cuando el código no cumple las reglas del lenguaje.

```
...
if edad >= 18
    print("Mayor de edad")
```

Python no puede ejecutar el programa porque falta `:` al final de la condición.

```
SyntaxError: expected ':'
```

> **Idea clave**
>
> Los errores de sintaxis suelen ser sencillos de localizar porque Python indica el punto en el que detecta el problema.

1.2. Errores en tiempo de ejecución

El programa comienza a ejecutarse, pero se produce una excepción.

```
numero = 10
divisor = 0

resultado = numero / divisor
```

Python mostrará un mensaje parecido a:

```
ZeroDivisionError: division by zero
```

La información mostrada forma parte del **traceback**, que indica por qué ha fallado el programa y en qué línea se produjo el error.

1.3. Errores lógicos

Son los errores en los que centrarse especialmente. **El programa se ejecuta sin excepciones, pero produce un resultado incorrecto**.

```
precios = [10, 20, 30]
total = 0

for precio in precios:
    total = precio

print(total)
```

El programa muestra:

```
30
```

pero el resultado correcto debería ser:

```
60
```

El problema está en:

```
total = precio
```

En cada iteración se sustituye (y no actualiza) el valor anterior. La operación correcta sería:

```
total += precio
```

> **Importante**
>
> En un error lógico Python no tiene por qué mostrar ningún mensaje. El programa puede terminar aparentemente bien aunque el resultado sea incorrecto.

3.4.2 2. Una metodología para localizar errores

Depurar no consiste en modificar líneas al azar hasta que el programa funcione. Es preferible seguir un proceso qeu podría ser

```
1. Reproducir el fallo
       ↓
2. Determinar el resultado esperado
       ↓
3. Si es necesario reducir los datos de entrada
       ↓
4. Localizar la zona sospechosa y observar variables
       ↓
5. Añadir print(), logging o colocar un breakpoint
       ↓
6. Avanzar paso a paso hasta identificar la causa
       ↓
7. Corregir y volver a probar
```

2.1. Determinar el resultado esperado

Si tiene:

```
notas = [7, 8, 5, 9]
```

la media esperada es `7.25`: antes de depurar debe saber qué resultado debería obtenerse.

2.2. Reproducir el fallo

Es importante *disponer de datos que hagan aparecer el error*. Los datos pequeños y fáciles de comprobar manualmente son ideales.

2.3. Formular hipótesis

Por ejemplo:

> Creo que el contador se reinicia en cada iteración.

La depuración consistirá en comprobar si esa hipótesis es cierta.

Depurar es comprobar

Una buena depuración intenta responder preguntas como:

- ¿Qué valor tiene esta variable?
- ¿Qué rama del `if` se está ejecutando?
- ¿Cuántas veces entra en este bucle?
- ¿Dónde cambia por primera vez el valor esperado?

3.4.3 3. Primera aproximación: depuración con `print()`

La técnica más sencilla consiste en añadir mensajes temporales al programa.

```
numeros = [10, 20, 30]

total = 0

for numero in numeros:
    total = numero
    print("numero:", numero)
    print("total:", total)

print("Resultado:", total)
```

Salida:

```
numero: 10
total: 10
numero: 20
total: 20
numero: 30
total: 30
```

Puede comprobar que `total` no acumula los valores.

3.1. Mensajes claros

Es mejor:

```
print("total =", total)
```

que:

```
print(total)
```

También puede utilizar f-strings:

```
print(f"numero={numero}, total={total}")
```

3.2. Inspeccionar tipos

Algunos errores se deben a tipos inesperados.

```
edad = input("Edad: ")

print(edad)
print(type(edad))
```

Resultado:

```
20
<class 'str'>
```

Aunque el usuario escriba un número, `input()` devuelve una cadena.

3.3. Limitaciones de `print()`

`print()` es útil, pero tiene limitaciones:

- obliga a modificar el código y es fácil olvidar eliminar mensajes tras la depuración.
- no permite detener el programa ni permite avanzar instrucción a instrucción.
- resulta incómodo en programas grandes.

3.4.4 4. Usando el debugger pdb y puntos de ruptura

Python incorpora la instrución `breakpoint()`. Cuando la ejecución alcanza esa línea, el programa se detiene y abre el depurador interactivo `pdb`.

```
numeros = [10, 20, 30]
total = 0

for numero in numeros:
    breakpoint()
    total += numero

print(total)
```

Podemos ejecutar el programa desde Linux:

```
python3 programa.py
```

Cuando se alcance el punto de interrupción aparecerá:

```
(Pdb)
```

4.1. Comandos básicos de `pdb`

Comando	Función
`p expresion`	Mostrar una variable o expresión
`n`	Ejecutar la siguiente línea
`s`	Entrar en una función
`c`	Continuar hasta el siguiente breakpoint
`l`	Mostrar código alrededor de la línea actual
`q`	Salir del depurador

4.2. Ejemplo

```
def calcular_descuento(precio, porcentaje):
    descuento = precio * porcentaje / 100
    precio_final = precio - descuento
    return precio_final


precio = 100
porcentaje = 20

breakpoint()

resultado = calcular_descuento(precio, porcentaje)

print(resultado)
```

Podemos consultar:

```
(Pdb) p precio
100
```

o:

```
(Pdb) p porcentaje
20
```

> **Diferencia entre `n` y `s`**
>
> `n` ejecuta una llamada a función sin entrar en ella.
>
> `s` entra en la función para depurarla línea a línea.

3.4.5 5. Depurar en Visual Studio Code

Para depurar Python con VS Code necesita primero instalar la extensión oficial de **Python Debugger** de Microsoft.

Y como ejercicio práctico vamos a depurar este script:

Supongamos este programa:

```
def calcular_media(lista):
    suma = 0

    for x in lista:
        suma = x

    return suma / len(lista)

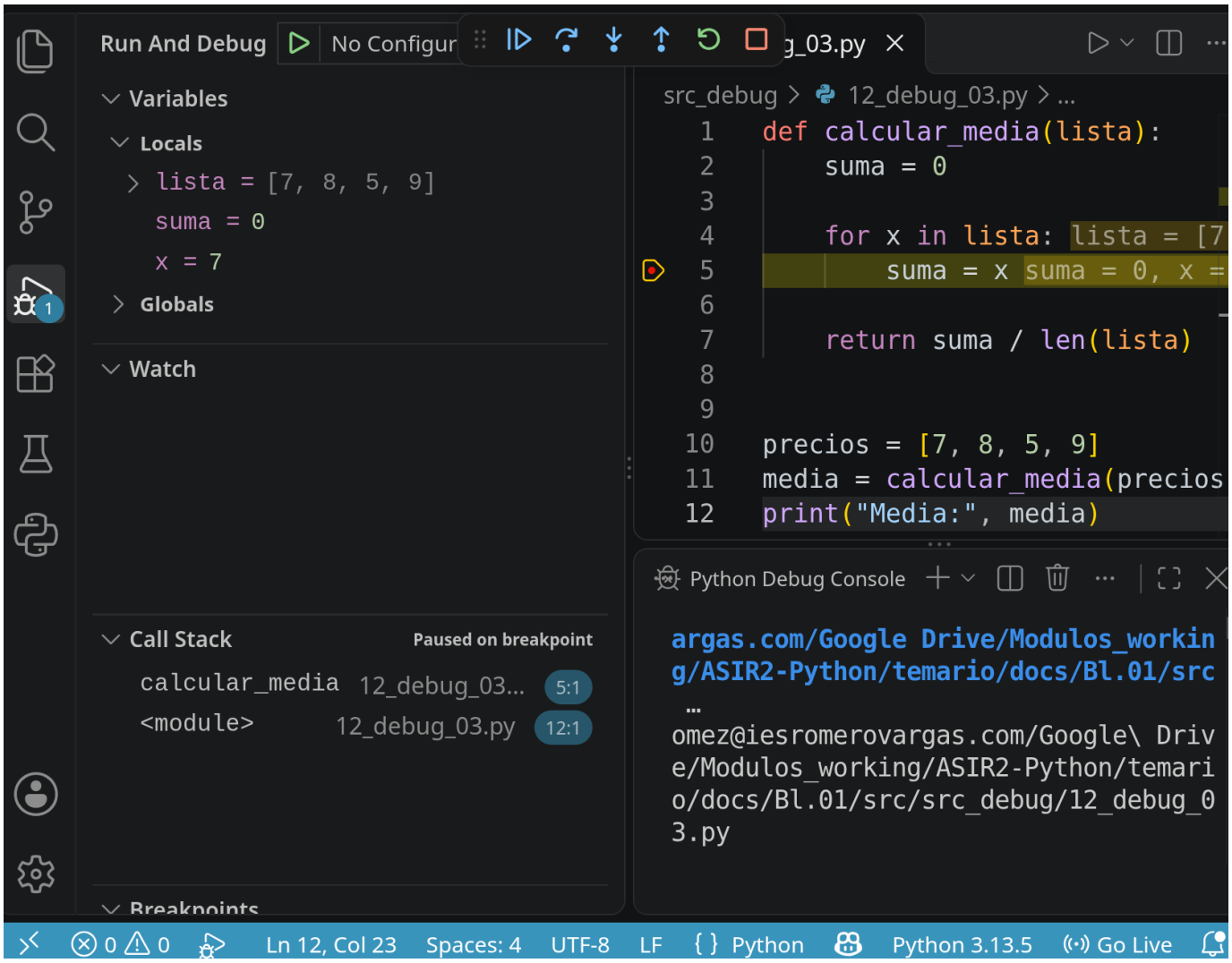
precios = [7, 8, 5, 9]
media = calcular_media(precios)
print("Media:", media)
```

Para colocar un **breakpoint** haga clic a la izquierda del número de línea y aparecerá un punto rojo. Para este ejemplo se situa en la línea

```
suma = x
```

5.1. Iniciar la depuración

Acceda el menú superior **Run**, seleccione **Start Debugging** y elija **Python Debugger** y **Python file**. El programa se detiene en el breakpoint.



5.2. Controles del depurador

- "Continue" (F5): Continúa la ejecución hasta el siguiente breakpoint.
- "Step Over" (F10): Ejecuta la línea actual y pasa a la siguiente sin entrar en funciones.
- "Step Into" (F11): Entra dentro de una función llamada en la línea actual.
- "Step Out" (Shift + F11): Continúa hasta salir de la función actual.

> **Qué control usar**
>
> Si sospechas que una función contiene el error, utiliza **Step Into**.
> Si ya confías en esa función, utiliza **Step Over**.

5.3. Inspección de variables

Cuando el programa se detiene, VS Code muestra a la izquierda el panel **Variables**. En el ejemplo anterior podemos observar:

```
lista = [7, 8, 5, 9]
suma = 0
x = 7
```

Si pulsa F5 en la siguiente iteración se observa:

```
suma = 7
x = 8
```

Y pulsando F5 de nuevo:

```
suma = 8
x = 5
```

El comportamiento revela el problema: suma va tomando el valor de x en cada iteración y no el acumulado. La instrucción debería ser:

```
suma += x
```

5.4. Breakpoints condicionales

Un bucle puede ejecutarse cientos de veces. Puede crear un punto de ruptura que solo se active cuando se cumpla una condición.

Por ejemplo, si el bucle itera sobre una lista muy grande de números:

```
for numero in numeros:
    procesar(numero)
```

basta **editar** el breakpoint, por ejemplo `numero == 50` o `numero < 0`

> **Caso de uso**
>
> Si un error solo aparece al procesar el usuario con identificador `153`, no tiene sentido detenerse en los 152 usuarios anteriores.

5.5. Pila de llamadas

El panel **Call Stack** muestra la cadena de funciones que ha llevado hasta el punto actual.

```
def aplicar_iva(precio):
    return precio * 1.21

def calcular_total(precios):
    total = 0
    for precio in precios:
        total += aplicar_iva(precio)
    return total

def generar_factura():
    precios = [10, 20, 30]
    return calcular_total(precios)

resultado = generar_factura()
print(resultado)
```

Durante la ejecución si situa un breakpoint en aplicar_iva se mostrará:

```
aplicar_iva()
calcular_total()
generar_factura()
main
```

Es decir, la cadena de llamadas es_

```
main
 ↓
generar_factura()
 ↓
calcular_total()
 ↓
aplicar_iva()
```

3.4.6 6. Errores lógicos frecuentes

6.1. Acumuladores mal actualizados

Ya mostrado previamente, el código incorecto es

```
total = 0
for precio in precios:
    total = precio      # Error, no acumula
```

y lo correcto es sustituir `total = precio` por `total += precio`

6.2. Ejercicio: acumulador mal iniciado

Este ejemplo es un contador de números pares que no funciona correctamente. Revisar y corregir **usando el depurador**

```
numeros = [3, 2, 6, 23, 16, 12, 67, 2]
for numero in numeros:
    contador = 0
    if numero % 2 == 0:
        contador += 1
```

6.3. Ejercicio: Sustituir en lugar de añadir

Este ejemplo crea una lista con los valores superiores a un límite establecido. Corregir **usando el depurador**:

```
LIMITE = 40
temperaturas = [36, 41, 47, 29, 44, 23, 39, 40, 12]
resultado = []  # lista con valores superiores a LIMITE

for t in temperaturas:
    if t > LIMITE:
        resultado = [numero]
```

6.4. Errores en valores límite

En el caso anterior hay una selectiva

```
    if t > LIMITE:
```

Si la temperatura LIMITE debe cumplir la condición debería ser:

```
    if t >= LIMITE:
```

Para comprobar límites es útil probar los valores previos y siguientes del límite, en el caso anterior probar que funciona bien con los valores

```
39
40
41
```

Idea final

Depurar muchas veces consiste en encontrar el primer punto en el que el comportamiento real del programa deja de coincidir con el comportamiento esperado.

4. Utiles

4.1 Python y venv

4.1.1 1. Necesidad de entornos virtuales

Puede consultar detalles sobre entornos virtuales en este enlace: A Guide to Python's Virtual Environments

Pero básicamente se pretende:

- Poder trabajar con proyectos con distintas versiones de lenguaje de programación.
- Poder tener en cada proyecto distintas versiones de los módulos que se importen.
- Poder reproducir el entorno de trabajo en otro equipo a partir de un fichero de texto con los requisitos.
- No añadir a la configuración global del equipo módulos que son específicos de cada proyecto.

4.1.2 2. Paso 1: crear el entorno virtual

Cree una carpeta para el proyecto y sitúese en ella:

```
clases@d12:~$ mkdir proyecto
clases@d12:~$ cd proyecto/
clases@d12:~/proyecto$
```

El comando para crear un entorno virtual en Python es

```
python3 -m venv <nombre_directorio_entornovirtual>`
```

Es habitual darle al directorio donde se almacena toda la información del entorno virtual siempre el mismo nombre, por ejemplo en este caso `venv` (o `.venv`).

```
clases@d12:~/proyecto$ python3 -m venv venv
clases@d12:~/proyecto$ ls -l
total 4
drwxr-xr-x 5 clases clases 4096 dic  1 20:12 venv
clases@d12:~/proyecto$
```

Warning

Es posible que al intentar crear el entorno virtual le solicite instalar el módulo `venv` si no estuviese instalado.

> On Debian/Ubuntu systems, you need to install the `python3-venv` package using the following command: `sudo apt install python3.11-venv`

Para ello:

```
clases@d12:~/proyecto$ sudo apt install python3.11-venv
[sudo] contraseña para clases:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Configurando python3.11-venv (3.11.2-6)
clases@d12:~/proyecto$
```

4.1.3 3. Paso 2: activar el entorno virtual

Crear el entorno no implica que se esté usando. Para activarlo debe ejecutar este comando:

```
clases@d12:~/proyecto$ source venv/bin/activate
(venv) clases@d12:~/proyecto$
```

> **Warning**
>
> Tenga en cuenta que la activación desde un entorno Windows es diferente:
>
> ```
> c:\proyecto\>.venv\Scripts\activate
> ```
>
> *Fuente:* Setting up a Python virtual environment on Windows for absolute beginners

4.1.4 3. Paso 3: Instalar un módulo

En el siguiente ejemplo (que resuelve un sistema de ecuaciones) se usa el módulo `numpy` que **no está instalado por defecto**.

El contenido del fichero `main.py`:

```
import numpy as np

# solve(a, b) : Devuelve la solución del sistema de ecuaciones lineales
# con los coeficientes de la matriz a y los términos independientes
# de la matriz b.
# Sistema de dos ecuaciones y dos incógnitas
# x + 2y = 1
# 3x + 5y = 2

a = np.array([[1, 2], [3, 5]])
b = np.array([1, 2])

print(np.linalg.solve(a, b))
# Resultado [-1. 1.]
```

Recién creado el entorno no debe haber ningún módulo instalado por el usuario. El comando `pip freeze` no muestra nada a la salida:

```
(venv) clases@d12:~/proyecto$ pip freeze
(venv) clases@d12:~/proyecto$
```

Si intenta ejecutat el script `main.py` da el error `ModuleNotFoundError` al no encontrar el módulo `numpy`:

```
(venv) clases@d12:~/proyecto$ python3 main.py
Traceback (most recent call last):
  File "/home/usuario/CIBER-PPS/bloque_03_programacion_segura/docs/tema_01_venv/src/numpy/main.py", line 14, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
(venv) clases@d12:~/proyecto$
```

Instale el módulo `numpy` en el entorno virtual utilizando la herramienta `pip`:

```
(venv) clases@d12:~/proyecto$ pip install numpy
Collecting numpy
...
Successfully installed numpy-2.2.0
(venv) clases@d12:~/proyecto$ pip freeze
numpy==2.2.0
(venv) clases@d12:~/proyecto$
```

Ahora puede probar de nuevo el ejemplo. El resultado será similar a:

```
(venv) clases@d12:~/proyecto$ python3 main.py
[-1. 1.]
(venv) clases@d12:~/proyecto$
```

Compruebe ahora la salida del comando `pip freeze`. Observará que se ha instalado en el entorno virtual `numpy` (y todos aquellos otrso módulos que necesite).

4.1.5 4. Paso 4: Salir del entorno

Para salir del entorno se usa el comando `deactivate`:

```
(venv) clases@d12:~/proyecto$ deactivate
clases@d12:~/proyecto$
```

Puede intentar ejecutar el script anterior uan vez desactivado y observar que no funciona:

```
clases@d12:~/proyecto$ python3 main.py
Traceback (most recent call last):
  File "/home/usuario/CIBER-PPS/bloque_03_programacion_segura/docs/tema_01_venv/src/numpy/main.py", line 1, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
clases@d12:~/proyecto$
```

Si activa de nuevo el entorno se mantienen los módulos instalados previamente y puede ejecutar de nuevo el script, etc.

```
clases@d12:~/proyecto$ source venv/bin/activate
(venv) clases@d12:~/proyecto$ python3 main.py
[-1. 1.]
(venv) clases@d12:~/proyecto$
```

4.1.6 5. Uso de fichero de requisitos

Si quiere almacenar los requisitos del proyecto puede generar un fichero con el listado de los módulos (y sus versiones) usando `pip freeze` y redirigir la salida a un fichero:

```
(venv) clases@d12:~/proyecto$ pip freeze > requirements.txt
(venv) clases@d12:~/proyecto$
```

Si quiere trabajar en otro equipo y recrear los mismos módulos basta, una vez creado y activado el nuevo entorno, ejecutar la instalación con `pip` de esos módulos a partir del fichero `requirements.txt` con la opción `-r`:

```
pip3 install -r requirements.txt
```

```
clases@d12:~/proyecto-sedeA$ ls -a
requirements.txt
clases@d12:~/proyecto-sedeA$ python3 -m venv venv
clases@d12:~/proyecto-sedeA$ source venv/bin/activate
(venv) clases@d12:~/proyecto-sedeA$ pip freeze
(venv) clases@d12:~/proyecto-sedeA$ pip install -r requirements.txt
Collecting numpy==2.2.0
  Using cached numpy-2.2.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.4 MB)
Installing collected packages: numpy
Successfully installed numpy-2.2.0
(venv) clases@d12:~/proyecto-sedeA$
```

4.1.7 6. Borrar el entorno virtual

Si desea borrar el entorno virtual basta con borrar el directorio `venv` creado dentro del proyecto:

```
(venv) clases@d12:~/proyecto$ deactivate
clases@d12:~/proyecto$ rm -rf venv/
clases@d12:~/proyecto$
```

4.1.8 7. Ejemplos de uso de entornos virtuales

> **Note**
>
> Depende del sistema operativo, módulos previamente instalados, etc puede que algunos de los módulos de estos ejemplos no necesiten instalación.

7.1. FastAPI

Puede ser que en algún momento trabaje con el framework de Python FastAPI. En ese caso necesita el módulo `fastapi` y adicionalmente un servidor de aplicaciones como `uvicorn`.

En este ejemplo sencillo el fichero `main.py` contiene:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

El proceso es similar al presentado previamente: crear un entorno virtual, iniciarlo

```
clases@d12:~$ mkdir app
clases@d12:~$ cd app
clases@d12:~/app$ python3 -m venv venv
clases@d12:~/app$ source venv/bin/activate
(venv) clases@d12:~/app$ nano main.py
(venv) clases@d12:~/app$
```

e instalar los módulos. En este caso lso módulos necesarios son `fastapi` y `uvicorn`:

```
(venv) clases@d12:~/app$ pip freeze
(venv) clases@d12:~/app$ pip install fastapi
Collecting fastapi
...
Successfully installed fastapi-0.104.1 idna-3.6
...
(venv) clases@d12:~/app$ pip install uvicorn
Collecting uvicorn
...
Installing collected packages: h11, click, uvicorn
Successfully installed click-8.1.7 h11-0.14.0 uvicorn-0.24.0.post1
(venv) clases@d12:~/app$
```

Ya puede lanzar el servidor `uvicorn` con la aplicación. Observe la forma de invocarla:

```
(venv) clases@d12:~/app$ uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['/home/clases/app']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [3380] using StatReload
INFO:     Started server process [3382]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Y ya puede acceder a la app en la URL `http://127.0.0.1:8000` (o a `http://127.0.0.1:8000/docs` para ver la documentación de la API creada).

```
clases@d12:~/app$ curl localhost:8000
{"message":"Hello World"}
clases@d12:~/app$ firefox localhost:8000 &
```

7.2. módulo psutil

Este es un ejemplo de uso del módulo `psutil` que permite obtener información el uso de los recursos del sistema.

```
import psutil

print ( " --- Monitorizando el Sistema Local ---" )

# 1. Uso de CPU
# interval =1 significa que mide durante 1 segundo
cpu_uso = psutil.cpu_percent(interval=1)

print(f" Uso actual de CPU : { cpu_uso } %")
```

Y el resultado esperado tras instalar el módulo:

$ python3 cpu.py --- Monitorizando el Sistema Local --- Uso actual de CPU : 6.6 % $

7.3. El módulo requests

Veamos un ejemplo sencillo del uso de la biblioteca `requests` que permite enviar solicitudes HTTP (como GET, POST, PUT, DELETE) a servidores web de forma sencilla, legible y rápida.

```
import requests

URL = "https://www.uca.es" # "https://www.uca.es/noexiste"
res = requests.get(URL)

print(res)
print(res.headers)
#print(res.text)
```

El resultado será similar a:

```
(venv) clases@d12:~/proyecto$ python prueba_request.py
<Response [200]>
```

```
{'Server': 'nginx', 'Date': 'Fri, 01 Dec 2023 19:47:02 GMT', 'Content-Type': 'text/html; charset=UTF-8', 'Content-Length': '20584', 'Connection': 'keep-
alive', 'X-Frame-Options': 'SAMEORIGIN', 'Link': '<https://www.uca.es/wp-json/>; rel="https://api.w.org/"', 'cache-control': 'max-age=0, must-revalidate', 'Co
ntent-Encoding': 'gzip', 'Vary': 'Accept-Encoding', 'X-Forwarded-Proto': 'https', 'age': '0', 'Accept-Ranges': 'bytes'}
(venv) clases@d12:~/proyecto$
```

7.4. Jupyter Notebook

> **Note**
>
> Se incluye este ejemplo para que observe que la instalación puede generar mas de 90 dependencias y ocupar entre 250MB y varios GB en función de la instalación.
>
> En estos casos se ve la importancia de no compartir el entorno virtual y ern su lugar compartir el fichero de requisitos.

Jupyter Notebook no es un "módulo" tradicional de Python en el sentido de una simple librería de funciones, sino un entorno web interactivo de desarrollo (IDE) de código abierto. Permite crear y compartir documentos web (archivos .ipynb) que combinan código ejecutable, ecuaciones matemáticas, visualizaciones y texto explicativo en un solo lugar.

El proceso de trabajo es similar: crear un nuevo directorio, un entorno virtual, se activa y se instala el módulo `notebook`. Tiene más información sobre esta herramienta en https://jupyter.org/.

En este ejemplo se mostrará Jupyter Notebook. También puede probar JupyterLab.

Este ejemplo muestra que un módulo puede tener dependencias de otros y en este caso instala todos los necesarios (en este caso alrededor de 90):

```
clases@d12:~$ mkdir notebook && cd notebook
clases@d12:~/notebook$ python3 -m venv venv
clases@d12:~/notebook$ source venv/bin/activate
(venv) clases@d12:~/notebook$ pip install notebook
Collecting notebook
...
clases@d12:~/notebook$ pip freeze | wc -l
90
(venv) clases@d12:~/notebook$
```

Una vez instalado, lanzar la aplicación con el comando `jupyter notebook`:

```
(venv) clases@d12:~/notebook$ jupyter notebook
[I 2023-12-01 22:00:04.835 ServerApp] Package notebook took 0.0000s to import
[I 2023-12-01 22:00:04.842 ServerApp] Package jupyter_lsp took 0.0066s to import
....
[I 2023-12-01 22:00:05.071 ServerApp] Serving notebooks from local directory: /home/clases/notebook
[I 2023-12-01 22:00:05.071 ServerApp] Jupyter Server 2.11.1 is running at:
[I 2023-12-01 22:00:05.071 ServerApp] http://localhost:8888/tree?token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
[I 2023-12-01 22:00:05.071 ServerApp] http://127.0.0.1:8888/tree?token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
[I 2023-12-01 22:00:05.071 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2023-12-01 22:00:05.084 ServerApp]

 To access the server, open this file in a browser:
 file:///home/clases/.local/share/jupyter/runtime/jpserver-2652-open.html
 Or copy and paste one of these URLs:
 http://localhost:8888/tree?token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
 http://127.0.0.1:8888/tree?token=208c5ce93f58f946e491a0c25f0ded05eaae1180f805863e
```

Nos muestra los mensajes para poder acceder a Jupyter Notebook, aunque lo normal es que nos abra el navegador ya con esa URL. Y ya podemos crear un cuaderno y trabajar con ese entorno:

```
# Ejemplo notebook
In [1]: print("¡Hola Mundo!")
¡Hola Mundo!
```

Para salir pulsar `Ctrl-C` en el terminal.

4.1.9 8. Enlaces

- A Guide to Python's Virtual Environments
- FastAPI: https://fastapi.tiangolo.com/
- Python requests. La librería para hacer peticiones http en Python: https://j2logo.com/python/python-requests-peticiones-http/

- Módulo psutil: Módulo psutil](https://psutil.readthedocs.io/stable/)
- Uso de jupyter online: https://jupyter.org/try
- Google Colab: https://colab.research.google.com/

4.2 Introducción a `logging`

> **Estado Borrador**
>
> Este documento está en estado borrador pendiente de completar.

`print()` es útil durante el desarrollo, pero no es la herramienta adecuada para registrar permanentemente el funcionamiento de una aplicación.

Python incluye el módulo `logging`

```
import logging

logging.basicConfig(level=logging.DEBUG)

logging.debug("Inicio del programa")

numero = 10

logging.debug("numero=%s", numero)
```

Salida aproximada:

```
DEBUG:root:Inicio del programa
DEBUG:root:numero=10
```

4.2.1 1. Niveles de logging

Nivel	Uso habitual
`DEBUG`	Información detallada para depuración
`INFO`	Funcionamiento normal
`WARNING`	Situación inesperada que permite continuar
`ERROR`	Error que afecta a una operación
`CRITICAL`	Error grave que puede impedir continuar

Ejemplos:

```
logging.debug("Valor de contador: %s", contador)
logging.info("Usuario autenticado correctamente")
logging.warning("Espacio en disco inferior al 10%%")
logging.error("No se pudo abrir el fichero")
logging.critical("No se puede conectar con la base de datos")
```

4.2.2 2. Configurar logging

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(levelname)s - %(message)s"
)
```

Puede incluir fecha y hora:

```
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
```

4.2.3 3. Logs en fichero

```
import logging

logging.basicConfig(
    filename="aplicacion.log",
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Inicio del programa")
```

consulta

En Linux puede consultar el archivo con

```
cat aplicacion.log
less aplicacion.log
tail -f aplicacion.log
```

4.2.4 4. Logging dentro de funciones

```
...

def calcular_media(notas):
    logging.debug("Notas recibidas: %s", notas)

    suma = 0
    for nota in notas:
        logging.debug("Procesando nota=%s", nota)
        suma += nota
        logging.debug("Suma acumulada=%s", suma)

    media = suma / len(notas)
    logging.debug("Media calculada=%s", media)
    return media
```

Los logs permiten reconstruir el comportamiento del programa.

4.2.5 5. `print()` frente a `logging`

Característica	`print()`	`logging`
Fácil de usar	Sí	Sí
Depuración rápida	Sí	Sí
Niveles de importancia	No	Sí
Guardar en fichero	Manual	Sí
Fecha y hora	Manual	Sí
Activar/desactivar mensajes	Difícil	Fácil
Adecuado para aplicaciones reales	No	Sí

4.2.6 6. Combinar logging y depurador

Las dos herramientas se complementan.

```
logging
   ↓
detectar una zona problemática
   ↓
breakpoint
   ↓
inspeccionar variables
   ↓
localizar la causa
```

4.3 Introducción a Python: argumentos en línea de comandos

Cuando ejecuta un programa desde la terminal puede pasarle **argumentos** para modificar su comportamiento sin tener que editar el código. Esta técnica es habitual en las herramientas de administración de sistemas Linux.

Por ejemplo:

```
$ ping -c 4 8.8.8.8
$ ls -l /var/log
$ python3 analizar.py --origen servidor1 --verbose
```

En el último comando, `analizar.py` es el script, `--origen` es una opción que recibe el valor `servidor1` y `--verbose` es un indicador que activa un comportamiento.

4.3.1 1. Tipos de argumentos

Considera este comando:

```
$ python3 herramienta.py consultar --producto disco --verbose
```

Elemento	Tipo	Descripción
`consultar`	Argumento posicional	Normalmente se identifica por su posición y, salvo configuración adicional, es obligatorio.
`--producto disco`	Opción con valor	Se reconoce por su nombre (`--producto`) y recibe el valor `disco` .
`--verbose`	Indicador o *flag*	Basta con escribirlo para activar una funcionalidad. No necesita un valor a continuación.

Aunque Python permite consultar los argumentos directamente mediante `sys.argv` , para scripts que necesitan varios argumentos resulta más cómodo utilizar `argparse` , un módulo de la biblioteca estándar que no requiere instalación adicional.

> **¿Argumentos o parámetros?**
>
> En el uso cotidiano se emplean ambos términos. Y se usa **opciones** cuando lleven nombres como `-p` o `--producto` .

4.3.2 2. Uso básico módulo `argparse`

`argparse` permite declarar qué argumentos acepta un script y se ocupa de tareas habituales:

- Comprobar si se han proporcionado los argumentos obligatorios.
- Intentar convertir sus valores al tipo indicado (`int` , `float` , etc.).
- Mostrar mensajes de error cuando los argumentos no cumplen esas condiciones.
- Generar automáticamente una ayuda con `-h` o `--help` .

> **Una limitación importante**
>
> `argparse` no comprueba automáticamente todas las reglas del programa. Por ejemplo, puede verificar que una cantidad sea un entero, pero **no que sea mayor que cero**. Esa comprobación se debería hacer en la aplicación.

2.1. Ejemplo: argumentos posicionales

Se crea un script denominado `uso_argparse.py` , basado en una pequeña calculadora. Por ahora solo recibirá y mostrará los datos.

uso_argparse.py

```python
import argparse

# Se crea el analizador de argumentos.
parser = argparse.ArgumentParser(
    description="Ejemplo de calculadora con argumentos posicionales."
)

# Se declaran tres argumentos posicionales.
parser.add_argument(
    "opera",
    help="Operación matemática: suma, resta, multiplica o divide."
)
parser.add_argument("num1", type=int, help="Primer operando entero.")
parser.add_argument("num2", type=int, help="Segundo operando entero.")

# Analiza lo que se ha escrito en la terminal.
args = parser.parse_args()

# Consulta de los argumentos mediante sus nombres.
print(f"Operación: {args.opera}")
print(f"Primer operando: {args.num1}")
print(f"Segundo operando: {args.num2}")
```

Ejecutw el programa:

```
$ python3 uso_argparse.py suma 10 5
Operación: suma
Primer operando: 10
Segundo operando: 5
```

¿Qué hace cada instrucción?

1. `argparse.ArgumentParser(...)` : crea el objeto que se encargará de analizar los argumentos. `description` añade una descripción a la ayuda.
2. `parser.add_argument(...)` : registra cada argumento admitido. Al no comenzar por `-`, estos tres argumentos son posicionales.
3. `type=int` : solicita convertir el argumento a entero. Si la conversión falla, `argparse` muestra un error.
4. `parser.parse_args()` : analiza los argumentos recibidos y devuelve un objeto de tipo `Namespace`.
5. `args.opera`, `args.num1` y `args.num2` : permiten acceder a los valores convertidos. **No se trata de un diccionario**, aunque sus atributos se consulten mediante nombres.

> **¿Es necesario indicar `type=str` ?**
>
> Si no se especifica `type`, `argparse` recibe el valor como una cadena de texto (`str`). P

Puede comprobar la validación automática con estos ejemplos de ejecución y observe los mensajes de error que genera:

```
$ python3 uso_argparse.py
$ python3 uso_argparse.py suma 10
$ python3 uso_argparse.py suma diez 5
$ python3 uso_argparse.py 10 5 suma
$ python3 uso_argparse.py suma 10 5 8
```

Las situaciones anteriores muestran, respectivamente, argumentos obligatorios ausentes, falta de un operando, un entero no válido, un orden incorrecto y argumentos sobrantes.

Por ejemplo, si falta el último argumento, aparecerá un mensaje similar al siguiente:

```
usage: uso_argparse.py [-h] opera num1 num2
uso_argparse.py: error: the following arguments are required: num2
```

Si se proporciona `diez` para `num1`, la conversión a `int` falla. Ante un error de uso, `argparse` muestra la explicación y, normalmente, termina el programa con **código de salida 2**.

> **En Linux**
>
> Para consultar el código de salida de la última orden ejecutada en Bash puedes utilizar `echo $?`. Comprueba la diferencia entre una ejecución correcta, `--help` y una ejecución con argumentos incorrectos.

2.3. La ayuda `--help`

La opción `-h` (equivalente a `--help`) se crea por defecto:

```
$ python3 uso_argparse.py --help
```

La ayuda presenta el uso del comando, la descripción del programa, los argumentos posicionales y sus explicaciones. La opción `-h` se muestra aunque no la hayamos declarado.

4.3.3 3. Opciones y valores permitidos

El siguiente ejemplo introduce novedades:

- `choices`: restringe los valores admitidos para un argumento.
- `-p` / `--producto` y `-c` / `--cantidad`: permiten **identificar opciones por su nombre en lugar de por su posición**.
- `--verbose` para ilustrar un indicador booleano.

gestion_inventario.py

```
import argparse

parser = argparse.ArgumentParser(
    description="Ejemplo de recepción de argumentos para un inventario."
)

# Argumento posicional con un conjunto de valores admitidos.
parser.add_argument(
    "accion",
    choices=["agregar", "eliminar", "consultar", "listar"],
    help="Operación que se desea realizar."
)

# Opciones con nombre: forma corta y forma larga.
parser.add_argument(
    "-p", "--producto",
    help="Nombre del producto."
)
parser.add_argument(
    "-c", "--cantidad",
    type=int,
    help="Cantidad de unidades."
)

# Indicador booleano: no recibe un valor.
parser.add_argument(
    "-v", "--verbose",
    action="store_true",
    help="Muestra información adicional."
)

args = parser.parse_args()

print(f"Acción: {args.accion}")
print(f"Producto: {args.producto}")
print(f"Cantidad: {args.cantidad}")

if args.verbose:
    print("Modo detallado activado.")
```

Ejemplos de invocaciones

```
$ python3 gestion_inventario.py agregar --producto disco --cantidad 3
Acción: agregar
Producto: disco
Cantidad: 3

$ python3 gestion_inventario.py agregar -c 3 -p disco -v
Acción: agregar
Producto: disco
Cantidad: 3
Modo detallado activado.

$ python3 gestion_inventario.py listar
Acción: listar
Producto: None
Cantidad: None
```

Observa que las opciones pueden utilizar su forma corta (`-p`) o larga (`--producto`) y que, en este ejemplo, **pueden aparecer en distinto orden**.

Cuando se omite una opción que espera un valor, su valor predeterminado es `None`, salvo que indiquemos otra cosa. En cambio, `args.verbose` vale `False` cuando no aparece `-v` y `True` cuando sí aparece.

Si se intenta ejecutar `python3 gestion_inventario.py modificar`, `argparse` rechazará la acción porque `modificar` no pertenece a la lista definida en `choices`.

3.2. Argumentos opcionales y obligatorios

Las opciones con nombre son opcionales por defecto. Podemos asignarles un valor predeterminado mediante `default`. Si modifica el código:

```
parser.add_argument(
    "-c", "--cantidad",
    type=int,
    default=1,
    help="Cantidad de unidades (por defecto: 1)."
)
```

También existe `required=True` para obligar a utilizar una opción, pero conviene reservarlo para los casos en que resulte necesario. Por ejemplo, la cantidad podría hacer falta para `agregar`, pero no para `listar`.

3.3. Validaciones que debe programar

Es necesario validar cuando los argumentos son válidos **individualmente**, pero no cumplen las condiciones de nuestra aplicación.

```
# se añade..
if args.accion == "agregar":
    if args.producto is None:
        parser.error("La acción 'agregar' requiere --producto.")

    if args.cantidad is None:
        parser.error("La acción 'agregar' requiere --cantidad.")

    if args.cantidad <= 0:
        parser.error("La cantidad debe ser mayor que cero.")
```

`parser.error(...)` muestra un mensaje de uso y termina la ejecución con un error.

4.3.4 7. Para ampliar información

- Documentación oficial de `argparse` (Python).
- Tutorial oficial de `argparse`.
- Introducción a `argparse` (recurso complementario).

4.4 Pyinstaller

> **Estado Borrador**
>
> Este documento está en estado borrador pendiente de completar.

> PyInstaller bundles a Python application and all its dependencies into a single package. The user can run the packaged app without installing a Python interpreter or any modules. PyInstaller supports Python 3.8 and newer, and correctly bundles many major Python packages such as numpy, matplotlib, PyQt, wxPython, and others.
>
> However, it is not a cross-compiler; to make a Windows app you run PyInstaller on Windows, and to make a Linux app you run it on Linux, etc.
>
> https://pyinstaller.org/en/stable/

4.4.1 Instalacion

Entorno virtual

```
clases@d13:~/src/test$ python3 -m venv venv
clases@d13:~/src/test$ source venv/bin/activate
(venv) clases@d13:~/src/test$
```

Instalación

```
(venv) clases@d13:~/src/test$ pip install pyinstaller
Collecting pyinstaller

...

Successfully installed altgraph-0.17.5 packaging-26.3 pyinstaller-6.22.3 pyinstaller-hooks-contrib-2026.7 setuptools-84.0.0
(venv) clases@d13:~/src/test$
```

4.4.2 Uso pyinstaller

```
(venv) clases@d13:~/src/test$ pyinstaller --onefile hola.py
30 INFO: PyInstaller: 6.22.3, contrib hooks: 2026.7
30 INFO: Python: 3.13.5

...

4462 INFO: Build complete! The results are available in: src/test/dist
(venv) clases@d13:~/src/test$ ls -l dist/
total 7944
-rw-r--r-- 1 clases clases 8130752 sep 18 13:44 hola
(venv) clases@d13:~/src/test$
```

4.4.3 Ejecución resultado

```
(venv) clases@d13:~/src/test$ chmod +x dist/hola
(venv) clases@d13:~/src/test$ ./dist/hola
Hola Mundo
(venv) clases@d13:~/src/test$
```

4.4.4 Otras formas de instalación

Dependiendo del uso que le vaya a dar a PyInstaller existen tres formas principales de proceder. En este apartado se ha optado por una de ellas:

1. Opción estándar para desarrollo: Entorno Virtual. Si está trabajando en un proyecto específico, lo ideal es usar un entorno virtual local.

```
python3 -m venv .venv
source .venv/bin/activate
pip install pyinstaller
```

2. Opción recomendable para aplicaciones independientes: `pipx` instala herramientas CLI de Python en entornos aislados automáticamente sin interferir con el sistema.

```
sudo apt install pipx
pipx install pyinstaller
```

3. Instalar mediante el gestor de paquetes del sistema (apt): Si está disponible en los repositorios de tu distribución, puedes instalarlo a nivel de sistema mediante APT.

```
sudo apt update
sudo apt install pyinstaller
```